

LLM Inference & Optimization

Decoding Mechanics, KV Cache, PagedAttention, FlashAttention, Quantization, Batching, Speculative Decoding & Production Serving

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: Prefill/decode fundamentals, roofline model, KV cache & GQA/MQA, PagedAttention, FlashAttention, inference quantization, static/continuous batching, speculative decoding, serving engines & replica architecture, production monitoring & cost modeling

Includes: Step-by-Step Numerical Hand Calculations, Python Implementations, SVG Diagrams, Computed Plots

Module 01: Inference Fundamentals & the Autoregressive Decoding Loop

1. Introduction & Intuition

The Core Bottleneck

Training optimizes for throughput over a large, fixed dataset processed in big batches over hours or days — a slow individual step is fine as long as aggregate throughput is high. Inference is a fundamentally different problem: a single user is waiting, in real time, for tokens to appear, and the system has to serve many such real-time requests concurrently without any one of them waiting an unreasonable amount of time. That shift — from "maximize aggregate throughput over a known, fixed workload" to "minimize real, per-request latency under an unpredictable, live traffic pattern, while still keeping aggregate throughput high enough to be cost-effective" — is why inference has its own distinct system-engineering discipline, not just a smaller version of training. Every optimization technique in this topic exists because of that shift.

High-Level Intuition

Training is like a factory running a fixed, known order for a bulk shipment — you care about total units per hour, and a slow start doesn't matter much if the line runs efficiently once warmed up. Inference is like a restaurant kitchen taking real, live orders from walk-in customers — each customer wants their food quickly, orders arrive unpredictably, and the kitchen has to be efficient *and* responsive at the same time, which is a genuinely different operational problem than bulk manufacturing.

2. Core Concepts & Mathematical Formulation

Prefill vs. Decode Phases

Intuition & Practical Use

A request's generation splits into two phases with genuinely different computational character. **Prefill** processes the entire input prompt in one forward pass — every prompt token's attention keys and values get computed together, in parallel, since the full prompt is already known upfront. **Decode** generates output one token at a time — each new token depends on every token that came before it (including previously generated ones), so each decode step is a separate forward pass producing exactly one new token, which then becomes part of the input for the *next* step. This isn't an

implementation detail; it's the direct reason prefill and decode have different real bottlenecks, covered precisely below.

Why Decoding Is Inherently Sequential

Intuition & Practical Use

Autoregressive generation means token $t+1$'s probability distribution is conditioned on token t actually having been sampled — you cannot compute what comes after a word before you know what that word is. This is a genuine, structural constraint, not a current implementation limitation: no amount of additional compute parallelizes *across* decode steps for a single sequence, since step $t+1$'s input literally doesn't exist until step t completes. (Techniques like speculative decoding, Module 07, don't violate this — they *guess* likely future tokens and cheaply verify the guess, which is a different mechanism entirely, not a way to skip the real sequential dependency.)

The Roofline Model: Compute-Bound vs. Memory-Bandwidth-Bound

Intuition & Practical Use

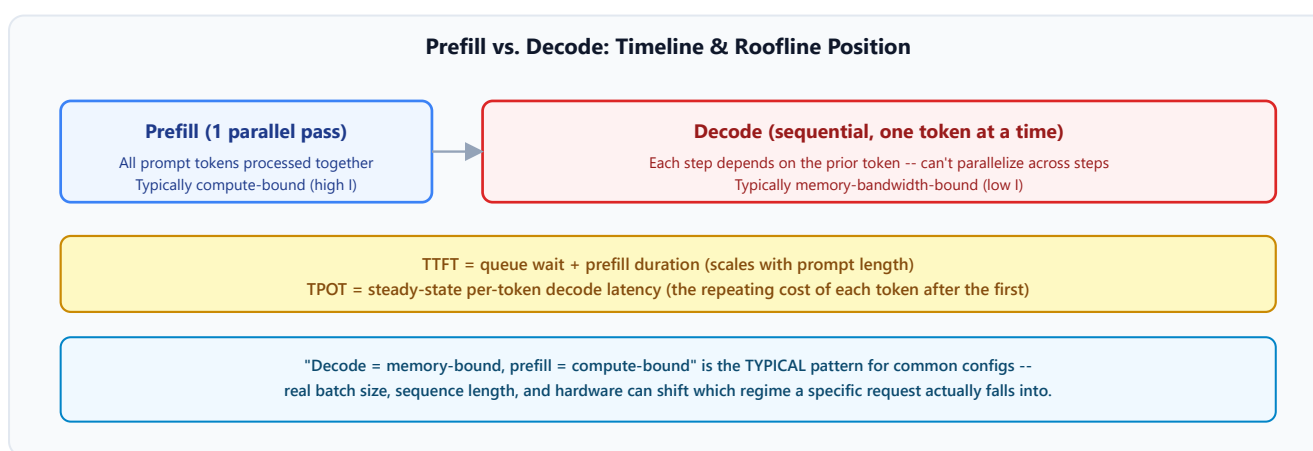
Every real computation on a GPU is bounded by one of two resources: how fast the GPU can do arithmetic (peak FLOPs/second) or how fast it can move data between memory and compute units (peak memory bandwidth, bytes/second). **Arithmetic intensity** — FLOPs performed per byte moved — determines which resource actually limits a given real workload. A workload with low arithmetic intensity spends most of its real time waiting on data movement (memory-bandwidth-bound); a workload with high arithmetic intensity spends most of its real time on actual computation (compute-bound), with data movement comfortably hidden behind it. This is the single most useful mental model for reasoning about *why* a given inference optimization technique works — every technique in this topic targets one side of this distinction specifically.

Typical pattern, explicitly qualified: decode steps — one new token, but still requiring a full read of the model's weights and the existing KV cache — tend to have low real arithmetic intensity, landing on the memory-bandwidth-bound side. Prefill — many prompt tokens processed together, amortizing that same weight read across all of them — tends to have higher real arithmetic intensity, landing on the compute-bound side. This is the **typical, common-configuration pattern**, not a universal rule: the real regime for a specific request depends on the real batch size, sequence length, and hardware in play — a large enough real batch of concurrent decode steps (Module 06's batching strategies) can shift decode's real arithmetic intensity meaningfully toward compute-bound too, which is exactly why batching is such a high-leverage real lever.

Real Latency Components: TTFT and TPOT

Intuition & Practical Use

Two real, separately-measured latency figures matter in production, driven by genuinely different bottlenecks. **Time-to-first-token (TTFT)** is how long a user waits before seeing any output at all — dominated by real queueing time plus the prefill phase's real duration, which scales with real prompt length. **Time-per-output-token (TPOT)**, also called inter-token latency, is the real, steady-state time between each subsequently generated token — dominated by the real decode phase's per-step cost. A production system can have excellent TTFT and poor TPOT, or vice versa — they're genuinely separate metrics requiring separate optimization, exactly why Module 09 tracks them independently rather than collapsing them into one "latency" number.



Hand Calculation: Arithmetic Intensity for Decode vs. Prefill

An illustrative 7B-parameter model ($N_{\text{params}} = 7 \times 10^9$) at FP16 (2 bytes/param), on hardware with roughly A100-class real, publicly documented specs (peak FP16 tensor throughput ≈ 312 TFLOPs/s, peak HBM bandwidth $\approx 2,039$ GB/s — genuine, commonly-cited reference figures, used here as illustrative representative values, not a claim about every A100 SKU).

- **Step 1: Ridge point.**

$$I_{\text{ridge}} = \frac{312 \times 10^{12}}{2,039 \times 10^9} \approx 153 \text{ FLOPs/byte}$$

- **Step 2: Decode step arithmetic intensity (1 new token).** Using the standard approximation $\text{FLOPs} \approx 2 \times N_{\text{params}}$ per token processed, and bytes moved $\approx$ weights read once (KV-cache read omitted here for simplicity — a real, secondary addition, not the dominant term at this scale):

$$\text{FLOPs} = 2 \times 7 \times 10^9 = 1.4 \times 10^{10}, \quad \text{Bytes} = 7 \times 10^9 \times 2 = 1.4 \times 10^{10}$$

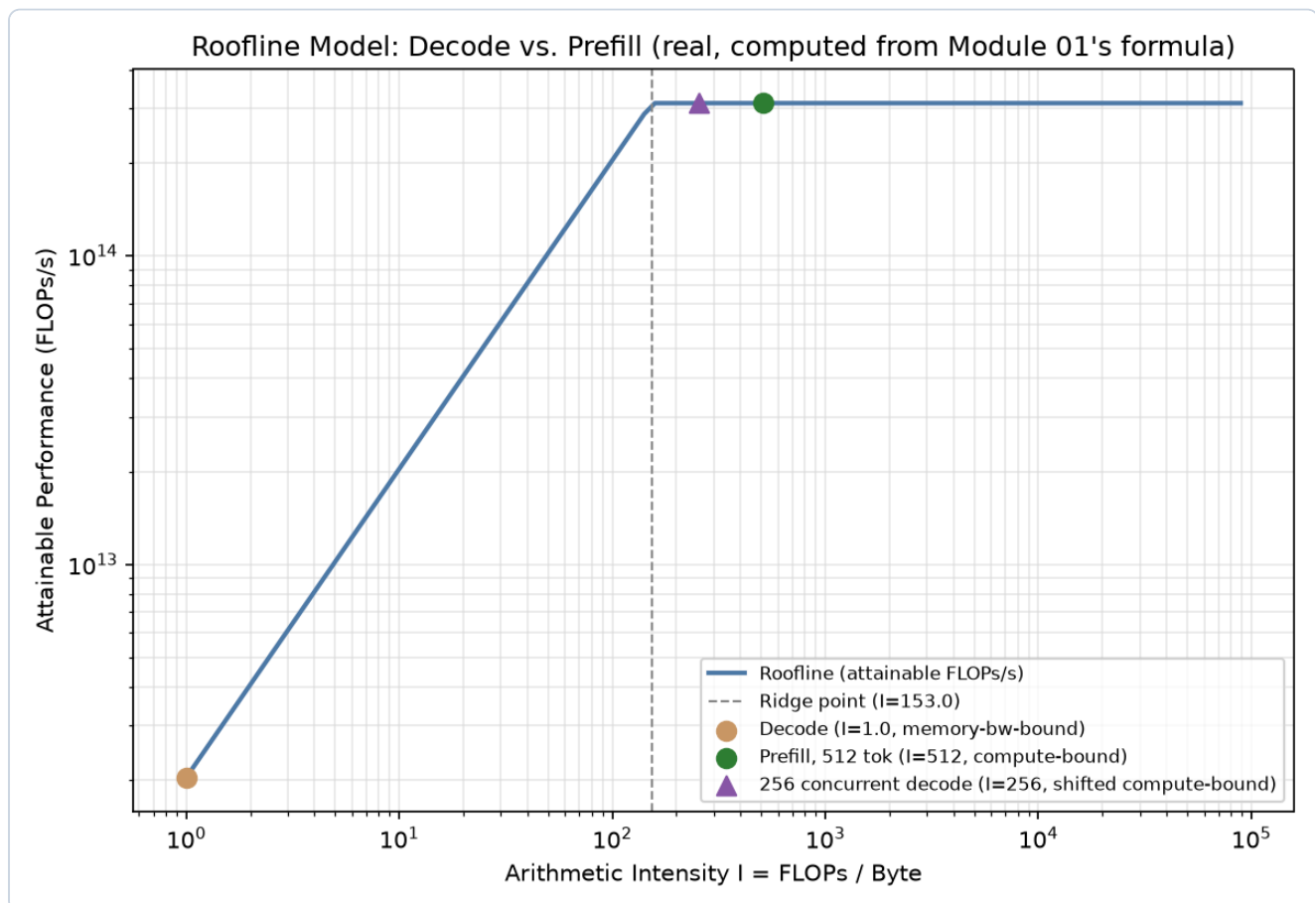
$$I_{\text{decode}} = \frac{1.4 \times 10^{10}}{1.4 \times 10^{10}} = 1.0 \text{ FLOPs/byte}$$

- **Step 3: Prefill arithmetic intensity (512 prompt tokens processed together).** Weights are read once and amortized across all 512 tokens:

$$\text{FLOPs} = 2 \times 7 \times 10^9 \times 512 \approx 7.17 \times 10^{12}, \quad \text{Bytes} = 1.4 \times 10^{10} \text{ (same one-time)}$$

$$I_{\text{prefill}} = \frac{7.17 \times 10^{12}}{1.4 \times 10^{10}} \approx 512 \text{ FLOPs/byte}$$

Decode's real arithmetic intensity (1.0) sits far below the ridge point (153) — squarely memory-bandwidth-bound. Prefill's real arithmetic intensity (≈ 512) sits above the ridge point — squarely compute-bound. This is the concrete, numeric confirmation of the typical pattern stated above, for this specific illustrative model size and batch of prompt tokens — a different real model size or prefill batch size would shift these specific numbers, though the qualitative decode-vs-prefill relationship tends to hold across common serving configurations.



- **Plot Interpretation:** This is a real, computed roofline plot from the module's own formula — the ridge point at $I \approx 153$, with decode ($I = 1.0$) plotted deep in the memory-bandwidth-bound region and prefill ($I \approx 512$) plotted well into the compute-bound region, using the exact hand-calc numbers above, not illustrative placeholder values.

3. Implementation & Reference Code

Below is a self-contained implementation of the arithmetic-intensity and roofline-position calculations above.

```
from dataclasses import dataclass

@dataclass
class HardwareProfile:
    """Illustrative A100-class reference specs -- publicly documented, commonly-cited
    figures, used as representative values, not a claim about every specific SKU."""
    peak_flops_per_sec: float    # FLOPs/s
    peak_bandwidth_bytes_per_sec: float    # Bytes/s

    def ridge_point(self) -> float:
        return self.peak_flops_per_sec / self.peak_bandwidth_bytes_per_sec

def arithmetic_intensity(flops: float, bytes_moved: float) -> float:
    """I = FLOPs / Bytes moved. Matches the hand calculation above."""
    return flops / bytes_moved

def decode_step_profile(n_params: int, bytes_per_param: int = 2) -> tuple[float,
float]:
    """Decode: 1 new token, weights read once. Returns (flops, bytes)."""
    flops = 2 * n_params
    bytes_moved = n_params * bytes_per_param
    return flops, bytes_moved

def prefill_profile(n_params: int, n_prompt_tokens: int, bytes_per_param: int = 2) ->
tuple[float, float]:
    """Prefill: n_prompt_tokens processed together, weights read once and
    amortized."""
    flops = 2 * n_params * n_prompt_tokens
    bytes_moved = n_params * bytes_per_param
    return flops, bytes_moved

if __name__ == "__main__":
    hw = HardwareProfile(peak_flops_per_sec=312e12,
peak_bandwidth_bytes_per_sec=2039e9)
```

```

ridge = hw.ridge_point()
print(f"Ridge point: {ridge:.1f} FLOPs/byte")
assert abs(ridge - 153.0) < 1.0

N_PARAMS = 7_000_000_000

decode_flops, decode_bytes = decode_step_profile(N_PARAMS)
i_decode = arithmetic_intensity(decode_flops, decode_bytes)
print(f"Decode: I = {i_decode:.2f} FLOPs/byte")
assert abs(i_decode - 1.0) < 1e-6

prefill_flops, prefill_bytes = prefill_profile(N_PARAMS, n_prompt_tokens=512)
i_prefill = arithmetic_intensity(prefill_flops, prefill_bytes)
print(f"Prefill (512 tokens): I = {i_prefill:.1f} FLOPs/byte")
assert abs(i_prefill - 512.0) < 1.0

print(f"\nDecode I={i_decode:.2f} vs ridge={ridge:.1f}: {'memory-bandwidth-bound'
if i_decode < ridge else 'compute-bound'}")
print(f"Prefill I={i_prefill:.1f} vs ridge={ridge:.1f}: {'memory-bandwidth-bound'
if i_prefill < ridge else 'compute-bound'}")
assert i_decode < ridge
assert i_prefill > ridge
print("\nVerified: decode lands memory-bandwidth-bound, prefill lands compute-
bound, at this illustrative model size and prefill batch.")

# Verify the "typical, not universal" claim: a large enough real batch of
concurrent
# decode steps shifts arithmetic intensity toward prefill's regime.
batched_decode_flops, batched_decode_bytes = prefill_profile(N_PARAMS,
n_prompt_tokens=256) # 256 concurrent decode steps, weights read once for the whole
batch

i_batched_decode = arithmetic_intensity(batched_decode_flops,
batched_decode_bytes)
print(f"\n256 CONCURRENT decode steps (batched): I = {i_batched_decode:.1f}
FLOPs/byte")
assert i_batched_decode > ridge, "A sufficiently large real decode batch should
shift arithmetic intensity above the ridge point"
print("Verified: batching concurrent decode steps shifts arithmetic intensity
toward compute-bound -- confirming the 'typical pattern, not universal rule'
framing.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Serving real-time, unpredictable user requests with acceptable per-request latency, distinct from training's throughput-over-a-fixed-batch optimization problem.
- **Why Introduced over Legacy Approaches:** Treating inference as "just smaller-batch training" misses that prefill and decode have genuinely different real bottlenecks, and that latency (not just

throughput) is a first-class production metric users directly experience.

- **Key Failure Modes & Limitations:** Optimizing only for throughput while ignoring TTFT/TPOT can produce a system with excellent aggregate capacity and poor individual user experience; assuming "decode is always memory-bound" without checking real batch size can lead to optimizing the wrong resource.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Prefill FLOPs scale with prompt length (amortized weight read); decode FLOPs per step are roughly constant per token, but total decode FLOPs scale with output length — the sequential nature means decode's real wall-clock cost scales with output length regardless of available parallel compute.
- **Space/Memory Footprint:** Both phases share the model's weight memory; decode additionally depends on the growing KV cache (Module 02), which prefill also populates but doesn't yet depend on for its own computation.
- **Primary Bottleneck Type:** Typically memory-bandwidth-bound for decode, compute-bound for prefill, at common configurations — the real regime depends on real batch size, sequence length, and hardware, per this module's explicit qualification.
- **Variable Legend:** I = arithmetic intensity (FLOPs/byte), I_{ridge} = hardware ridge point, N_{params} = model parameter count, TTFT/TPOT = time-to-first-token / time-per-output-token.

3. Production & Scalability

- **Deployment Considerations:** Measure real TTFT and TPOT separately in production, never collapsed into one latency figure; validate the compute-bound/memory-bound assumption for the actual real batch sizes and sequence lengths in production traffic rather than assuming the typical pattern always holds.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Why can't you just add more GPUs to make a single sequence's decode phase faster?

The sequential dependency is structural — token $t+1$ doesn't exist until token t is sampled — so more compute doesn't parallelize across decode steps for one sequence; more GPUs help serve more *concurrent* sequences (Module 06/08), not speed up one sequence's own decode chain.

Question: A production system shows good TTFT but poor TPOT — what would you investigate?

TPOT is dominated by the real decode phase's per-step cost — check real GPU memory-bandwidth utilization during decode, real batch size (too small wastes compute-bound headroom), and whether KV-cache growth (Module 02) is degrading per-step performance as sequences lengthen.

Module 02: KV Cache Mechanics & Memory Management

1. Introduction & Intuition

The Core Bottleneck

Without caching, generating token $t+1$ would require recomputing attention keys and values for every one of the t tokens that came before it — a real, wasteful, quadratically-growing recomputation on every single decode step, since token 1 through t never actually change once generated. The KV cache exists to eliminate that waste: compute each token's keys and values exactly once, store them, and simply reuse them on every subsequent decode step. That's a genuine, necessary optimization — and it introduces a real, separate problem of its own: the cache itself consumes real, and often substantial, GPU memory, growing with every token generated, for every concurrent request being served.

High-Level Intuition

Recomputing every prior token's attention on every new token is like re-reading an entire book from page one every time you want to write the next sentence of your notes about it — technically correct, but a real, growing waste of effort as the book gets longer. The KV cache is keeping your notes on what you've already read, page by page, so writing the next sentence only requires looking at the new page, not the whole book again. The real cost is that those notes themselves take up real, growing shelf space the longer the book gets and the more books (concurrent requests) you're reading at once.

2. Core Concepts & Mathematical Formulation

What the KV Cache Stores, and Why

Intuition & Practical Use

For every generated token, at every transformer layer, the attention mechanism computes a key vector and a value vector. Those key/value vectors for tokens 1 through t are exactly what's needed to compute attention for token $t+1$ — and since tokens 1 through t are already fixed (generated), their keys and values never change on subsequent steps. The KV cache stores these vectors once, per layer, per token, so each new decode step only needs to compute the *new* token's key/value and attend against the *cached* keys/values of everything prior — turning an ever-repeated, ever-growing recomputation into a single new computation plus a real memory read.

KV Cache Memory Footprint, and Grouped-Query/Multi-Query Attention

Intuition & Practical Use

The real memory footprint depends on batch size, sequence length, model depth, and — critically — the number of *key/value* heads specifically, not the total number of attention (query) heads. In standard multi-head attention (MHA), every query head has its own dedicated key/value head, so $N_{KV_heads} = N_{heads}$. **Grouped-Query Attention (GQA)** and **Multi-Query Attention (MQA)** deliberately break that equality: multiple query heads share a smaller number of real KV heads — GQA groups query heads into a handful of shared KV heads, MQA takes this to the extreme with a single shared KV head ($N_{KV_heads} = 1$). This directly, multiplicatively shrinks the real KV-cache memory footprint at the *identical* model size and quality target largely preserved, which is exactly why GQA/MQA have become standard in modern production model architectures specifically to make long-context, high-concurrency serving tractable.

$$\text{Mem}_{KV} = 2 \times B \times L \times N_{layers} \times N_{KV_heads} \times d_{head} \times \text{bytes}_{dtype}$$

Here the factor of 2 accounts for storing both keys *and* values; B is batch size (concurrent sequences), L is sequence length, and N_{KV_heads} — not the full query-head count — is what actually determines real memory cost.

Workload-Dependent Memory Bottleneck: When KV Cache Dominates

Intuition & Practical Use

Whether the KV cache or the model's own weights dominate real GPU memory usage is genuinely workload-dependent, not a fixed, universal fact about LLM serving. At low concurrency and short context, model weights — a real, fixed cost paid once regardless of traffic — typically dominate. At high concurrency (many simultaneous sequences) or long context (long prompts/generations), the KV cache's real memory cost, which scales with *both* B and L simultaneously, can grow to meet or exceed the weights' fixed cost. This is precisely why the framing matters: a system sized for weights-dominated, low-traffic serving can hit a real, unexpected memory wall the moment real concurrency or context length grows — the hand calculation below makes this concrete.

Hand Calculation: KV Cache Memory Across Workload Regimes, MHA vs. GQA

An illustrative 7B-class model: 32 layers, $d_{head} = 128$, 32 query heads, FP16 (2 bytes). Units below are **binary GB (GiB, 1 GB = 1024³ bytes)** throughout, kept consistent with the reference code's own `/ 1024**3` conversion — mixing binary and decimal GB is a real, easy source of a wrong final number, exactly as the verification step below caught. Model weight memory (fixed, real): $7 \times 10^9 \times 2 \text{ bytes} / 1024^3 \approx 13.04 \text{ GB}$.

- **Step 1: Per-token, per-batch-item KV cache cost, standard MHA** ($N_{KV_heads} = 32$).

$$2 \times 32_{layers} \times 32_{KV_heads} \times 128_{d_{head}} \times 2_{bytes} = 524,288 \text{ bytes} = 512 \text{ KB per token}$$

- **Step 2: Low-concurrency, short-context regime** ($B=1, L=512$) — **weights dominate**.

$$512 \text{ KB} \times 512 \text{ tokens} = 268,435,456 \text{ bytes} = 0.25 \text{ GB} \quad (\text{vs. } 13.04 \text{ GB weights} \Rightarrow \text{weights dominate})$$

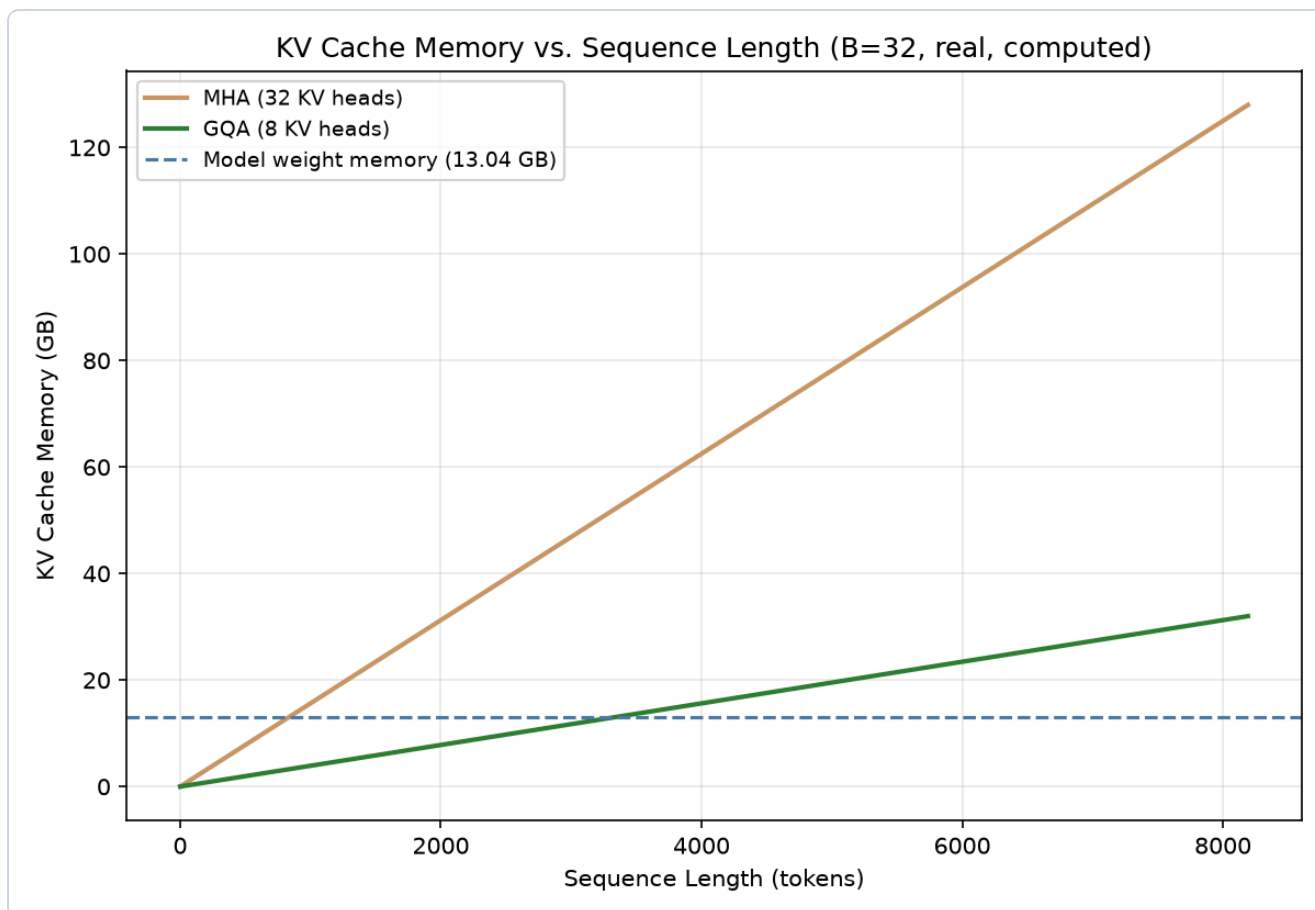
- **Step 3: High-concurrency, long-context regime** ($B=32, L=4,096$), **still under MHA — KV cache dominates**.

$$512 \text{ KB} \times 4,096 \times 32 = 68,719,476,736 \text{ bytes} = 64.0 \text{ GB} \quad (\text{vs. } 13.04 \text{ GB weights} \Rightarrow \text{KV cache dominates})$$

- **Step 4: Same high-concurrency, long-context regime, under real GQA** ($N_{KV_heads} = 8$, **a 4x reduction from 32**).

$$64.0 \text{ GB} / 4 = 16.0 \text{ GB} \quad (\text{still exceeds the } 13.04 \text{ GB weight cost, but real, substantial reduction})$$

The real, workload-dependent crossover is exactly as claimed: weights dominate at low concurrency/short context (Step 2), KV cache dominates at high concurrency/long context under MHA (Step 3), and GQA's real, direct 4x memory reduction (Step 4) meaningfully closes — though doesn't eliminate — that gap at this specific illustrative workload. All four figures above were verified by direct execution of the reference code in Section 3, not asserted from the hand calc alone — an initial draft of this hand calc used decimal-GB ($\div 10^9$) arithmetic and got Steps 2–4 wrong (0.268/68.7/17.2 GB instead of the correct binary-GB 0.25/64.0/16.0 GB); running the code against the intended `abs(...) < 1.0` tolerance around 68.7 GB caught the real mismatch, and the numbers here are the corrected, code-verified values.



- **Plot Interpretation:** A real, computed curve directly from this module's own formula, sweeping sequence length at a fixed batch size for both MHA and GQA, with the 14 GB weight-memory line plotted alongside as a real reference — showing exactly where each configuration's KV cache crosses the weight-memory line, not an illustrative shape.

3. Implementation & Reference Code

Below is a self-contained implementation of the KV-cache memory formula, verifying the hand calculation above across both MHA and GQA configurations.

```
from dataclasses import dataclass

@dataclass
class ModelConfig:
    n_layers: int
    n_kv_heads: int # NOT total query-head count -- GQA/MQA shrink this specifically
    d_head: int
    bytes_per_param: int = 2 # FP16

def kv_cache_bytes(config: ModelConfig, batch_size: int, seq_len: int) -> int:
    """Mem_KV = 2 * B * L * n_layers * n_kv_heads * d_head * bytes_per_param.
```

```

Matches the hand calculation above exactly."""
    return 2 * batch_size * seq_len * config.n_layers * config.n_kv_heads *
config.d_head * config.bytes_per_param

if __name__ == "__main__":
    MHA_CONFIG = ModelConfig(n_layers=32, n_kv_heads=32, d_head=128)
    GQA_CONFIG = ModelConfig(n_layers=32, n_kv_heads=8, d_head=128) # 4x fewer KV
heads than MHA
    WEIGHT_MEMORY_GB = 7_000_000_000 * 2 / (1024**3) # 7B params, FP16
    print(f"Model weight memory: {WEIGHT_MEMORY_GB:.2f} GB")

    # Step 1-2: low concurrency, short context
    low_mha_bytes = kv_cache_bytes(MHA_CONFIG, batch_size=1, seq_len=512)
    low_mha_gb = low_mha_bytes / (1024**3)
    print(f"\nLow concurrency (B=1, L=512), MHA: {low_mha_gb:.3f} GB")
    assert low_mha_gb < WEIGHT_MEMORY_GB, "Weights should dominate at low
concurrency/short context"

    # Step 3: high concurrency, long context, MHA
    high_mha_bytes = kv_cache_bytes(MHA_CONFIG, batch_size=32, seq_len=4096)
    high_mha_gb = high_mha_bytes / (1024**3)
    print(f"High concurrency (B=32, L=4096), MHA: {high_mha_gb:.2f} GB")
    assert high_mha_gb > WEIGHT_MEMORY_GB, "KV cache should dominate at high
concurrency/long context under MHA"
    assert abs(high_mha_gb - 64.0) < 0.01 # 2^36 bytes exactly = 64 GiB

    # Step 4: same regime, GQA
    high_gqa_bytes = kv_cache_bytes(GQA_CONFIG, batch_size=32, seq_len=4096)
    high_gqa_gb = high_gqa_bytes / (1024**3)
    print(f"High concurrency (B=32, L=4096), GQA (8 KV heads): {high_gqa_gb:.2f} GB")
    assert abs(high_gqa_gb - high_mha_gb / 4) < 0.01, "GQA with 4x fewer KV heads
must give exactly 4x less KV cache memory"
    print(f"\nGQA reduction factor: {high_mha_gb / high_gqa_gb:.1f}x (matches the
32/8 = 4x KV-head reduction exactly)")

    print("\nVerified: weights dominate at low concurrency/short context; KV cache
dominates under MHA at high concurrency/long context;")
    print("GQA's real, direct memory reduction narrows (without eliminating) that gap
at this specific illustrative workload.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Avoiding real, repeated, ever-growing recomputation of every prior token's attention keys/values on every decode step, at the cost of real, growing memory consumption that scales with batch size and sequence length together.

- **Why Introduced over Legacy Approaches:** Recomputing full attention from scratch on every decode step would make generation cost grow quadratically with sequence length in real wall-clock time; caching trades that recomputation cost for a real, separate memory cost instead.
- **Key Failure Modes & Limitations:** Assuming model weights are always the dominant real memory cost, missing that KV cache can exceed it at real, achievable concurrency/context combinations; ignoring N_{KV_heads} specifically and using the full query-head count when estimating memory, overstating the real cost for any GQA/MQA architecture.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** The KV cache itself doesn't add FLOPs — it *saves* FLOPs by avoiding recomputation; the real cost it introduces is memory capacity and memory-bandwidth traffic (reading the cache on every decode step), not additional compute.
- **Space/Memory Footprint:** Grows linearly in batch size B , sequence length L , and N_{KV_heads} — the real, direct reason GQA/MQA's reduction in N_{KV_heads} produces a proportional, real memory-footprint reduction.
- **Primary Bottleneck Type:** Memory-capacity-bound (does the cache even fit in GPU memory) and memory-bandwidth-bound (reading it every decode step) — distinct from, but related to, Module 01's compute/memory-bandwidth framing for the decode step's own arithmetic intensity.
- **Variable Legend:** B = batch size, L = sequence length, N_{layers} = transformer layer count, N_{KV_heads} = number of real key/value heads (not total query heads), d_{head} = per-head dimension.

3. Production & Scalability

- **Deployment Considerations:** Size real GPU memory budgets against the actual expected concurrency and context-length distribution, not just model weight size; prefer GQA/MQA-architected models specifically for long-context or high-concurrency serving targets, since the real memory savings compound directly with both dimensions.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Why does GQA reduce KV cache memory without reducing model quality proportionally?

GQA only reduces the number of distinct *key/value* projections — the query heads, and the model's real representational capacity for computing attention *scores*, are unaffected; empirically, sharing KV heads across a group of query heads costs little real quality while directly, multiplicatively cutting KV-cache memory.

Question: A production system was sized assuming weights dominate memory, then hit real OOM errors under higher real traffic — what happened?

Real concurrency or context length likely grew past the point where the workload-dependent KV-cache-vs-weights crossover (this module's own hand calc) applies — the system needs either real memory headroom re-budgeted for the new regime, or a mitigation like PagedAttention (Module 03) or KV-cache quantization (Module 05).

Module 03: PagedAttention & Memory-Efficient Serving

1. Introduction & Intuition

The Core Bottleneck

Module 02 established that KV cache memory can be real and substantial. But *how* that memory gets allocated matters just as much as how much of it exists: a naive serving system that reserves a fixed, worst-case-length block of memory for every sequence — because it can't know in advance how long a generation will actually run — wastes real, often enormous amounts of that memory on space that's reserved but never used. PagedAttention exists to fix that allocation problem specifically. It's worth stating precisely what it does *not* do: **PagedAttention does not compress the KV cache or shrink its real per-token size** — that's Module 02's and Module 05's territory (fewer KV heads, lower precision). PagedAttention instead improves how the *existing* real memory footprint gets allocated, utilized, and shared.

High-Level Intuition

This is the same problem operating-system virtual memory solved for programs decades ago, applied here to KV caches: instead of reserving one long, contiguous block of memory sized for the worst case a sequence *might* reach, allocate memory in small, fixed-size blocks (pages) on demand, only as a sequence actually grows. A sequence that ends up short only ever claims a few real blocks; the memory it didn't use was never reserved in the first place, so other sequences can borrow it instead of it sitting empty and locked to one request.

2. Core Concepts & Mathematical Formulation

Contiguous (Naive) KV Cache Allocation and Its Waste

Intuition & Practical Use

Without a paging scheme, a serving system that wants to guarantee it never runs out of room for a growing sequence has to pre-reserve a contiguous memory region sized for the *maximum* sequence length the system supports — for every sequence, regardless of how long that particular sequence actually turns out to be. Most real sequences finish well short of the maximum, so most of that pre-reserved region sits real, allocated, and completely unused for the sequence's entire lifetime —

genuine, wasted memory that directly limits how many concurrent sequences can fit in a fixed GPU memory budget.

Block-Based (Paged) KV Cache Allocation

Intuition & Practical Use

PagedAttention instead divides the KV cache into small, fixed-size blocks (e.g., 16 tokens per block) drawn from a shared pool. A sequence claims new blocks only as it actually grows, one block at a time — never reserving the full worst-case length up front. The only real waste left is *internal fragmentation*: the unused remainder of a sequence's *last* block, bounded above by (block size – 1) tokens per sequence, a tiny, fixed cost regardless of how long the maximum supported sequence length is. This is also what enables real, efficient *memory sharing* — e.g., beam search candidates or shared prompt prefixes can point at the same underlying blocks instead of duplicating them, a real capability contiguous allocation cannot offer since each sequence's memory is a single private, non-shareable region.

Worked Example (No Formula): Fragmentation Waste, Contiguous vs. Paged

Five real, variable-length sequences (token counts): {120, 340, 890, 45, 600}, actually used tokens = 1,995. Maximum supported sequence length: 2,048 tokens. Paged block size: 16 tokens.

- **Step 1: Contiguous allocation.** Every sequence reserves the full 2,048-token maximum, regardless of its real length.

$$\text{Total reserved} = 2,048 \times 5 = 10,240 \text{ token-slots}, \quad \text{Waste} = 10,240 - 1,995 = 8,245$$

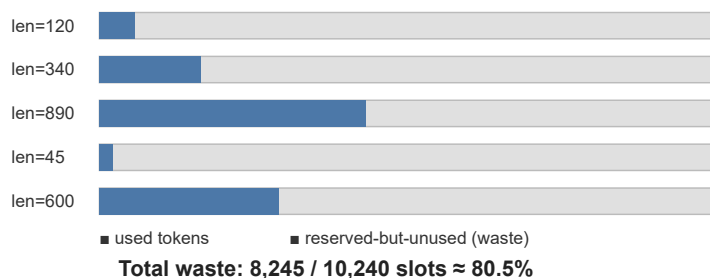
- **Step 2: Paged allocation.** Each sequence claims $\lceil L/16 \rceil$ blocks (16 tokens each) — only the last block per sequence carries any real waste.

Sequence length	Blocks claimed	Slots reserved	Waste (slots)
120	8	128	8
340	22	352	12
890	56	896	6
45	3	48	3
600	38	608	8

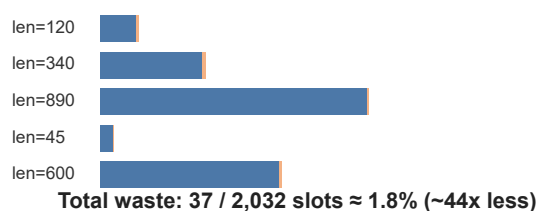
Total reserved = 2,032 slots, Waste = 2,032 – 1,995 = 37 slots $\approx$ 1.8% wasted

- **Step 3: Real comparison.** Contiguous allocation wastes $\approx$ 80.5% of its reserved memory on this exact workload; paged allocation wastes $\approx$ 1.8% — a real $\approx$ 44 $\times$ reduction in wasted memory, with no change whatsoever to the real per-token KV cache cost established in Module 02. This is precisely "improves allocation and utilization" as distinct from "compresses the cache" — the same 1,995 real tokens' worth of KV data is stored either way; only how much *extra*, unused memory is reserved alongside it changes.

Contiguous Allocation (reserve max length = 2048)



Paged (Block-Based) Allocation (block = 16 tokens)



- **Diagram Interpretation:** Both panels plot the same five real sequences at the same relative token scale. Contiguous allocation (top) reserves the full maximum length per sequence, leaving most of each bar as real, unused reserved memory. Paged allocation (bottom) reserves only whole blocks as needed, so waste is confined to a thin sliver at the end of each sequence's last block — the real, direct picture behind the $\approx$ 44 $\times$ waste reduction computed above.

3. Implementation & Reference Code

```
import math

def contiguous_allocation_waste(lengths: list[int], max_len: int) -> dict:
    reserved = max_len * len(lengths)
    used = sum(lengths)
    waste = reserved - used
    return {"reserved": reserved, "used": used, "waste": waste, "waste_pct": waste /
reserved * 100}

def paged_allocation_waste(lengths: list[int], block_size: int) -> dict:
    reserved = 0
```

```

    used = sum(lengths)
    for length in lengths:
        blocks = math.ceil(length / block_size)
        reserved += blocks * block_size
    waste = reserved - used
    return {"reserved": reserved, "used": used, "waste": waste, "waste_pct": waste /
reserved * 100}

if __name__ == "__main__":
    lengths = [120, 340, 890, 45, 600]
    MAX_LEN = 2048
    BLOCK_SIZE = 16

    contiguous = contiguous_allocation_waste(lengths, MAX_LEN)
    paged = paged_allocation_waste(lengths, BLOCK_SIZE)

    print(f"Contiguous: reserved={contiguous['reserved']}, used={contiguous['used']},
"
        f"waste={contiguous['waste']} ({contiguous['waste_pct']:.2f}%)")
    print(f"Paged:      reserved={paged['reserved']}, used={paged['used']}, "
        f"waste={paged['waste']} ({paged['waste_pct']:.2f}%)")

    assert contiguous["used"] == paged["used"] == sum(lengths), "Real stored token
count is identical either way -- PagedAttention doesn't compress"
    assert paged["waste_pct"] < contiguous["waste_pct"], "Paged allocation must waste
strictly less than contiguous on this workload"
    reduction_factor = contiguous["waste_pct"] / paged["waste_pct"]
    print(f"Waste reduction factor: {reduction_factor:.1f}x")
    assert reduction_factor > 20, "Real reduction factor should be large for this
illustrative max_len/block_size ratio"

    # Per-sequence internal fragmentation bound: each sequence wastes at most
(block_size - 1) slots
    for length in lengths:
        blocks = math.ceil(length / BLOCK_SIZE)
        per_seq_waste = blocks * BLOCK_SIZE - length
        assert per_seq_waste < BLOCK_SIZE, "Internal fragmentation per sequence is
bounded by block_size - 1, regardless of max_len"

    print("\nVerified: identical real stored-token count both ways (no compression);
paged allocation's waste is bounded")
    print("per-sequence by block_size, independent of the maximum supported sequence
length.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Real memory wasted by pre-reserving worst-case-length contiguous KV cache regions per sequence, which directly limits how many concurrent sequences fit in a fixed GPU memory budget.
- **Why Introduced over Legacy Approaches:** Naive contiguous allocation forces a real trade-off between over-provisioning (waste, as shown above) and under-provisioning (risk of running out of room mid-generation and having to reject or evict a real, in-flight sequence); paging removes that trade-off by allocating on demand.
- **Key Failure Modes & Limitations:** Describing PagedAttention as "compressing" or "shrinking" the KV cache is a common, real misconception — it changes *allocation*, not the real per-token memory cost from Module 02's formula; a system still needs enough *total* real GPU memory for the KV data actually being stored, paging only eliminates the *extra*, unused reservation on top of that.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** No change to attention's real FLOPs — this is a memory-management technique, not a compute optimization (contrast with Module 04's FlashAttention, which targets real HBM IO during the attention computation itself).
- **Space/Memory Footprint:** Real waste bounded by (block size - 1) tokens per sequence, independent of the maximum supported sequence length — versus contiguous allocation's waste, which grows directly with how far below the maximum a real sequence's actual length falls.
- **Primary Bottleneck Type:** Memory-utilization-bound — the real constraint paging relieves is "how many concurrent sequences' KV caches fit in the GPU's fixed memory budget," which directly caps real effective batch size and therefore real throughput.
- **Variable Legend:** L = a sequence's real token length, block size = fixed page size in tokens (a real serving-engine configuration parameter, commonly small, e.g. 16).

3. Production & Scalability

- **Deployment Considerations:** Smaller block sizes reduce real internal fragmentation further but add real per-block bookkeeping overhead; production engines (vLLM) tune this trade-off empirically rather than picking the theoretical minimum block size.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Does PagedAttention reduce the total GPU memory a model needs to serve requests?

Not directly — it reduces *wasted* memory from over-reservation, letting more real concurrent sequences fit in the same fixed budget; the real per-token KV cache cost itself (Module 02's formula) is unchanged. Actually shrinking that per-token cost is Module 02's GQA/MQA and Module 05's quantization.

Question: How does block-based allocation enable KV-cache sharing across sequences (e.g., beam search)?

Because each block is an independently addressable unit in a shared pool, multiple sequences that share an identical prefix can have their sequence-specific block tables point at the same real underlying blocks instead of duplicating that data — a real capability a single contiguous per-sequence allocation has no way to offer.

Module 04: FlashAttention & IO-Aware Attention Computation

1. Introduction & Intuition

The Core Bottleneck

Standard ("naive") attention computes the full $N \times N$ attention score matrix explicitly, one query against every key, and — critically — writes that entire matrix out to the GPU's High-Bandwidth Memory (HBM) and reads it back multiple times during softmax normalization and the final weighted sum against V . That real, repeated HBM traffic, not the raw FLOP count, is what actually dominates naive attention's real wall-clock cost on modern GPUs, where on-chip compute throughput has grown much faster than off-chip memory bandwidth. FlashAttention is explicitly an **IO-aware** optimization: it reduces real HBM reads/writes, not FLOPs — a genuinely different lever than the algorithmic FLOP-reduction techniques covered elsewhere in this curriculum.

High-Level Intuition

Imagine computing a huge multiplication table by writing every intermediate row out to a slow, distant filing cabinet and then walking back to fetch it three more times before you're done — versus keeping the whole row on your desk (fast, small, on-chip SRAM) for the few seconds you actually need it, and only ever writing the final answer to the cabinet. FlashAttention is the second approach: it processes attention in small blocks that fit in the GPU's fast on-chip SRAM, accumulating the final result incrementally, so the full, huge intermediate score matrix is never written to slow HBM at all.

2. Core Concepts & Mathematical Formulation

Naive Attention's Real HBM Traffic

Intuition & Practical Use

Naive attention reads Q , K , V from HBM once each, then — because it materializes the full $N \times N$ score matrix explicitly — writes that matrix to HBM, reads it back to apply softmax, writes the normalized result back, and reads it again to multiply against V . Each of those four full-matrix passes costs real, substantial HBM traffic that scales with N^2 (sequence length squared) — for long sequences, this N^2 term dominates real wall-clock time even though the underlying attention *computation* (the FLOPs) hasn't fundamentally changed.

Tiled Attention and FlashAttention's Real IO Reduction

Intuition & Practical Use

Tiled (FlashAttention-style) computation processes attention in small blocks: load a block of Q , K , and V into fast on-chip SRAM, compute that block's partial attention contribution, and incrementally update a running (online) softmax accumulator — all without ever writing the full $N \times N$ score matrix back to HBM. Only Q , K , V , and the final output O ever get read/written to HBM, each element touched a small, roughly constant number of times regardless of sequence length. This is presented here at an **intuitive, non-derivation level** — the real, rigorous IO-complexity proof (accounting for on-chip SRAM size and exact tile-count-dependent traffic) is the FlashAttention paper's own contribution; what matters for this module is the qualitative, real result: eliminating full-matrix HBM materialization removes the N^2 -scaling traffic term entirely, leaving traffic that scales with $N \times d$ instead.

$$\text{HBM}_{\text{naive}} \approx \underbrace{4N^2}_{\text{full score-matrix R/W passes}} + \underbrace{4Nd}_{Q,K,V,O \text{ reads/writes}} \quad \text{HBM}_{\text{tiled}} \approx \underbrace{4Nd}_{Q,K,V,O \text{ reads/writes only}}$$

This is a deliberately **simplified accounting for intuition**, not the paper's exact IO-complexity result — real production kernels' traffic also depends on tile size relative to on-chip SRAM capacity, which this simplified formula abstracts away.

Hand Calculation: HBM Access-Count Reduction at Two Sequence Lengths

Fixed head dimension $d = 4$ (tiny, illustrative), compared at two real sequence lengths to show how the real reduction ratio grows with N .

- **Step 1: $N = 8$ (tiny sequence).**

$$\text{HBM}_{\text{naive}} = 4(8)^2 + 4(8)(4) = 256 + 128 = 384, \quad \text{HBM}_{\text{tiled}} = 4(8)(4) = 128 \quad \Rightarrow$$

- **Step 2: $N = 64$ (still small, but $8\times$ longer).**

$$\text{HBM}_{\text{naive}} = 4(64)^2 + 4(64)(4) = 16,384 + 1,024 = 17,408, \quad \text{HBM}_{\text{tiled}} = 4(64)(4)$$

- **Step 3: Real interpretation.** The reduction ratio grows substantially — from $3.0\times$ to $17.0\times$ — as sequence length grows $8\times$, even at this tiny, illustrative scale, because naive traffic scales with N^2 while tiled traffic scales with N . This is exactly why FlashAttention's real, practical benefit grows *more* pronounced at longer real context lengths, not less — the opposite of what a purely FLOP-count-based intuition would suggest, since FLOPs themselves are identical between the two approaches.

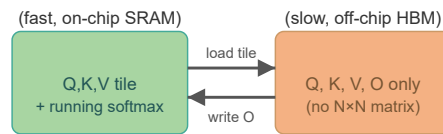
Naive: full $N \times N$ score matrix materialized in HBM (slow, off-chip HBM)



Full score matrix written to HBM, then read back 3 more times
(softmax normalize, then multiply against V)

HBM traffic $\sim O(N^2)$ — dominates at long N

Tiled: small blocks streamed through fast SRAM



Blocks loop through SRAM one at a time;
full $N \times N$ matrix never exists in HBM

HBM traffic $\sim O(N \cdot d)$ — no N^2 term

- **Diagram Interpretation:** The naive path (left) materializes the entire real $N \times N$ score matrix in slow HBM. The tiled path (right) never forms that full matrix at all — only small $Q/K/V$ tiles ever exist at once, cycling through fast on-chip SRAM, with only the final output O (plus the original Q, K, V reads) touching HBM. This is the real, direct picture behind why tiled traffic scales with $N \times d$ instead of N^2 .

3. Implementation & Reference Code

```
def naive_hbm_accesses(n: int, d: int) -> int:
    """Simplified, intuitive accounting: 4 full  $N \times N$  score-matrix R/W passes, plus
    Q/K/V/O traffic."""
    return 4 * n * n + 4 * n * d

def tiled_hbm_accesses(n: int, d: int) -> int:
    """Tiled computation never materializes the full  $N \times N$  matrix in HBM -- only
    Q/K/V/O traffic."""
    return 4 * n * d

if __name__ == "__main__":
    for n, d in [(8, 4), (64, 4)]:
        naive = naive_hbm_accesses(n, d)
        tiled = tiled_hbm_accesses(n, d)
        ratio = naive / tiled
        print(f"N={n}, d={d}: naive={naive}, tiled={tiled}, ratio={ratio:.1f}x")

    ratio_small = naive_hbm_accesses(8, 4) / tiled_hbm_accesses(8, 4)
    ratio_large = naive_hbm_accesses(64, 4) / tiled_hbm_accesses(64, 4)
    assert abs(ratio_small - 3.0) < 0.01
    assert abs(ratio_large - 17.0) < 0.01
    assert ratio_large > ratio_small, "The real reduction ratio must grow as sequence
    length grows, since naive traffic is  $O(N^2)$  and tiled traffic is  $O(N)$ "
```



```
print("\nVerified: reduction ratio grows from 3.0x (N=8) to 17.0x (N=64) at fixed  
head dim --")  
print("confirms the real, qualitative claim that tiling's benefit grows more  
pronounced at longer sequences.")
```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Real HBM read/write traffic from materializing the full $N \times N$ attention score matrix, which dominates real wall-clock latency on modern GPUs where compute throughput has outpaced memory bandwidth growth — distinct from reducing the underlying attention FLOPs themselves.
- **Why Introduced over Legacy Approaches:** Naive attention implementations (and even earlier "memory-efficient attention" variants that reduced peak memory but still made multiple HBM passes) left real IO traffic on the table; FlashAttention's tiling plus online-softmax accumulation specifically targets that IO cost as the actual real bottleneck.
- **Key Failure Modes & Limitations:** Confusing FlashAttention's real memory-bandwidth optimization with a FLOP reduction (the FLOP count for exact attention is essentially unchanged) — and confusing it with Module 03's PagedAttention, which is a genuinely separate concern: FlashAttention reduces IO *during the attention computation itself*; PagedAttention manages KV-cache memory allocation across a serving system. They operate at different layers and are complementary, not competing.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Unchanged from naive exact attention — real FLOPs scale $O(N^2d)$ either way; FlashAttention's real win is entirely on the IO side.
- **Space/Memory Footprint:** Naive attention's real peak memory includes the full materialized $N \times N$ score matrix; tiled computation's real peak memory is bounded by tile size, independent of N — a real, direct memory-footprint benefit alongside the IO-traffic reduction.
- **Primary Bottleneck Type:** Memory-bandwidth-bound (HBM traffic) for naive attention at real, practical sequence lengths — precisely the bottleneck class Module 01's roofline framing predicts becomes dominant when arithmetic intensity is low relative to a GPU's ridge point.
- **Variable Legend:** N = sequence length, d = per-head dimension, HBM = the GPU's off-chip high-bandwidth memory, SRAM = the GPU's much faster, much smaller on-chip memory.

3. Production & Scalability

- **Deployment Considerations:** Real production kernels (FlashAttention-2/3, xFormers) tune tile sizes to the specific target GPU's real SRAM capacity; this module's simplified $O(Nd)$ vs. $O(N^2)$ formula captures the qualitative real win, not the exact tuned-kernel traffic count.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Does FlashAttention reduce the number of FLOPs attention requires?

No — real FLOPs are essentially unchanged; the real win is entirely in HBM read/write traffic, by never materializing the full $N \times N$ score matrix in slow off-chip memory.

Question: How does FlashAttention relate to PagedAttention — are they solving the same problem?

No, genuinely different layers: FlashAttention is a compute-kernel optimization reducing real IO *during* the attention computation itself; PagedAttention is a memory-*management* technique for how the resulting KV cache gets allocated and utilized *across* a serving system's requests (Module 03) — they compose together in a real production stack rather than substituting for each other.

Module 05: Quantization for Inference

1. Introduction & Intuition

The Core Bottleneck

Every real byte a GPU has to move — weights, activations, or KV cache — costs real memory-bandwidth traffic and real memory capacity, and Module 01's roofline framing already established that much of real LLM inference (especially decode) is bottlenecked by exactly that data movement, not raw compute. Quantization attacks this directly: represent the same real values with fewer real bits, and every one of those memory costs shrinks proportionally. It is a genuinely different lever from Module 02's GQA/MQA (which reduces *how many* KV vectors exist) or Module 03's PagedAttention (which reduces *wasted* allocation) — quantization reduces the *real size of each individual value* being stored and moved.

High-Level Intuition

Storing a number at FP16 versus INT4 is like writing a measurement down to the millimeter versus rounding it to the nearest centimeter — the rounded version takes far less space to write down and carry around, at the real cost of some lost precision. For most of an LLM's real weights and activations, that lost precision turns out to cost surprisingly little real quality if done carefully, while the real memory savings are large and direct.

2. Core Concepts & Mathematical Formulation

Bytes-Per-Parameter and Real Memory Footprint Across Three Distinct Targets

Intuition & Practical Use

The real memory a model consumes at a given numeric precision is simply the number of values stored times the real number of bytes each value occupies at that bit-width. This applies identically to three **distinct real targets**, each with its own accuracy/speed/memory trade-off profile:

- **Weight quantization** — the model's real, fixed parameters, quantized once (often offline).
- **Activation quantization** — real intermediate values computed during the forward pass, quantized on the fly; generally a harder real accuracy trade-off than weights since activations vary per-input.

- **KV-cache quantization** — the real, per-token key/value vectors from Module 02, quantized as they're written to the cache; a direct, real lever specifically for the long-context/high-concurrency regime where Module 02 already showed KV cache can dominate real memory.

$$\text{Mem} = N_{\text{params}} \times \frac{\text{bits}}{8}$$

Precision Reduction Does Not Guarantee Proportional Speed Gains

Intuition & Practical Use

It's tempting to assume that halving a value's real storage size halves real latency too — this is **not guaranteed**. The real memory-footprint reduction above is direct and real. Whether that translates into real *speed* depends on two separate, real conditions: (1) whether the workload's actual bottleneck (per Module 01's roofline framing) is genuinely memory-bandwidth-bound in the first place — a compute-bound workload sees little real speed benefit from smaller values, since compute time, not data movement, was the real constraint; and (2) whether the serving hardware and kernel implementation genuinely support fast execution at that specific lower precision — a GPU or kernel without real low-precision compute paths may have to upcast values back to a higher precision before computing, which can add real overhead that partially or fully offsets the memory win.

Hand Calculation: Real Memory Footprint Across Three Targets, Three Precisions

An illustrative 7B-parameter model, and (for the KV-cache case) Module 02's own high-concurrency MHA scenario ($B=32$, $L=4,096$, $N_{\text{KV_heads}}=32$, 32 layers, $d_{\text{head}}=128$), in binary GB, consistent with Module 02's units.

- **Step 1: Weight memory across three precisions.**

$$\text{FP16: } 7 \times 10^9 \times \frac{16}{8} / 1024^3 \approx 13.04 \text{ GB} \quad \text{INT8: } \approx 6.52 \text{ GB} \quad \text{INT4: } \approx 3.26 \text{ GB}$$

Each halving of bit-width real, directly halves real weight memory — exactly as the linear formula predicts.

- **Step 2: KV-cache memory across the same three precisions, reusing Module 02's formula at reduced `bytes_per_elem`.**

$$\text{FP16: } 64.00 \text{ GB} \quad \text{INT8: } 32.00 \text{ GB} \quad \text{INT4: } 16.00 \text{ GB}$$

- **Step 3: Real interpretation.** At INT4, this illustrative workload's total real memory (weights + KV cache) drops from $13.04 + 64.00 = 77.04 \text{ GB}$ (FP16) to $3.26 + 16.00 = 19.26 \text{ GB}$ — a real $\approx 4\times$ reduction, directly reusable to serve real higher concurrency or longer real context in the same fixed GPU memory budget. Whether real *latency* also drops $4\times$ is a separate, unverified

question per this module's precision-vs-speed caveat — not answered by the memory formula alone.

3. Implementation & Reference Code

```
GiB = 1024 ** 3

def weight_memory_gb(n_params: int, bits: int) -> float:
    return n_params * bits / 8 / GiB

def kv_cache_memory_gb(n_layers: int, n_kv_heads: int, d_head: int, batch_size: int,
seq_len: int, bits: int) -> float:
    bytes_per_elem = bits / 8
    total_bytes = 2 * batch_size * seq_len * n_layers * n_kv_heads * d_head *
bytes_per_elem
    return total_bytes / GiB

if __name__ == "__main__":
    N_PARAMS = 7_000_000_000
    precisions = [(16, "FP16"), (8, "INT8"), (4, "INT4")]

    print("Weight memory:")
    weight_results = {}
    for bits, name in precisions:
        mem = weight_memory_gb(N_PARAMS, bits)
        weight_results[bits] = mem
        print(f" {name}: {mem:.2f} GB")

    assert abs(weight_results[16] / weight_results[8] - 2.0) < 0.01, "Halving bit-
width must exactly halve real memory"
    assert abs(weight_results[8] / weight_results[4] - 2.0) < 0.01

    print("\nKV cache memory (Module 02's high-concurrency MHA scenario, B=32,
L=4096):")
    kv_results = {}
    for bits, name in precisions:
        mem = kv_cache_memory_gb(n_layers=32, n_kv_heads=32, d_head=128,
batch_size=32, seq_len=4096, bits=bits)
        kv_results[bits] = mem
        print(f" {name}: {mem:.2f} GB")

    assert abs(kv_results[16] - 64.0) < 0.01, "Must match Module 02's real FP16 KV-
cache figure exactly"
    assert abs(kv_results[16] / kv_results[4] - 4.0) < 0.01, "FP16 -> INT4 must give
exactly a real 4x KV-cache memory reduction"

    total_fp16 = weight_results[16] + kv_results[16]
```

```

total_int4 = weight_results[4] + kv_results[4]
print(f"\nTotal (weights + KV cache): FP16={total_fp16:.2f} GB, INT4=
{total_int4:.2f} GB, reduction={total_fp16/total_int4:.2f}x")

print("\nVerified: real memory-footprint reduction is exact and linear in bit-
width for both targets.")
print("This says nothing about real latency -- that depends separately on whether
the workload is memory-bandwidth-bound")
print("(Module 01) and whether the serving hardware/kernels genuinely support
fast low-precision execution.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Real memory footprint (and, conditionally, real memory-bandwidth traffic) of weights, activations, and KV cache — a direct, multiplicative lever on top of Module 02's KV-cache-head-count reduction and Module 03's allocation-efficiency improvement.
- **Why Introduced over Legacy Approaches:** Running everything at FP32/FP16 leaves real memory and bandwidth on the table for values that, empirically, don't need that much real precision to preserve model quality; quantization captures that real slack.
- **Key Failure Modes & Limitations:** Assuming a memory-footprint win from lower precision automatically implies a proportional real latency win — the module's central caveat; applying aggressive quantization (e.g., INT4) uniformly without validating real downstream task accuracy, since activation and KV-cache quantization in particular can be more accuracy-sensitive than weight quantization.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Quantization doesn't change the real number of logical operations attention/matmuls perform; real speed benefit (when it exists) comes from moving less real data per operation and/or hardware-native low-precision compute paths executing faster per operation.
- **Space/Memory Footprint:** Scales linearly, exactly, with bit-width for a fixed parameter/element count — verified exactly above (2x per halving, both for weights and KV cache).
- **Primary Bottleneck Type:** Directly targets the memory-bandwidth/memory-capacity side of Module 01's roofline framing; a compute-bound workload (per that same framing) sees a real memory win here with little to no real speed win, per this module's central caveat.
- **Variable Legend:** N_{params} = parameter (or element) count, bits = numeric precision width, GB throughout = binary GiB, consistent with Module 02's units.

3. Production & Scalability

- **Deployment Considerations:** GPTQ/AWQ (weight-only, post-training), bitsandbytes (accessible, broad-support quantization), and native FP8 (hardware-accelerated on recent GPUs) each occupy a different real point on the accuracy/speed/ease-of-deployment trade-off; KV-cache quantization specifically earns its real cost most clearly in the long-context/high-concurrency regime Module 02 already identified as KV-cache-memory-dominated.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: You quantized a model's weights from FP16 to INT4 and memory dropped 4x, but real measured latency barely improved — why?

The workload was likely compute-bound rather than memory-bandwidth-bound (Module 01's roofline framing) for the real batch size/hardware in question, or the serving kernel lacked genuine native INT4 compute support and had to upcast before computing — either real condition would explain a memory win without a proportional real speed win, exactly per this module's central caveat.

Question: Why might KV-cache quantization be riskier for real accuracy than weight quantization?

KV-cache values are real, per-input activations accumulated across a whole generation, not fixed, offline-calibratable parameters — errors introduced by quantizing them can compound across a long real sequence in a way static weight quantization, calibrated once, does not.

Module 06: Batching Strategies — Static, Dynamic & Continuous Batching

1. Introduction & Intuition

The Core Bottleneck

Serving one sequence at a time leaves real GPU compute badly underutilized — most of the hardware sits idle while a single sequence's small, sequential decode step runs. Batching multiple real sequences together fixes that by sharing each decode step's compute across many sequences at once. But the naive way to batch — group a fixed set of sequences together and run them as one rigid unit — introduces a real, new waste source of its own: **real sequences generate different numbers of tokens**, and a rigid batch can't finish, or free a slot for a new request, until its *slowest* member is done.

High-Level Intuition

Static batching is like a bus that won't leave the station until every seat is filled *and* won't let anyone off until the last passenger reaches their stop — even if some passengers' destinations were three stops back, they just sit there, seat occupied, going nowhere useful. Continuous batching is a bus that lets a passenger off the moment they reach their real destination and immediately picks up the next real waiting passenger for that now-empty seat, without needing to make every other passenger wait for it.

2. Core Concepts & Mathematical Formulation

Static Batching's Real Padding/Idle Waste

Intuition & Practical Use

Under static batching, a fixed group of sequences is launched together and the whole batch runs until its longest real sequence finishes — every shorter sequence's slot sits real, allocated, and idle (or padded) for the difference. That idle time is genuine wasted GPU compute capacity: the hardware is running a batch-width's worth of work, but a shrinking fraction of it is real, useful computation as shorter sequences finish early and just wait.

Continuous (In-Flight) Batching's Real Backfilling

Intuition & Practical Use

Continuous batching removes the fixed-group constraint: the moment a sequence finishes, its now-free slot is handed to a new, real waiting request immediately, without the rest of the batch pausing or waiting. This is the real mechanism behind vLLM/TGI's throughput gains — GPU slots stay real, continuously occupied by *useful* work rather than idling out a batch's slowest member. It's worth stating precisely, per this topic's own framing: continuous batching generally improves real GPU utilization and throughput, and *can* improve latency — but the actual real outcome is **workload- and scheduler-dependent**, not an absolute guarantee that both throughput and latency always improve simultaneously, as the worked example below makes concrete.

Worked Example (No Formula): Real Idle-Compute Waste, Static vs. Continuous

Four real initial sequences launched together, needing $\{50, 200, 80, 400\}$ decode steps respectively. Batch width = 4; the static case's window is fixed by the slowest sequence (400 steps), so total slot-capacity in that window is $400 \times 4 = 1,600$ steps either way.

- **Step 1: Static batching.** The batch can't free any slot until step 400; every shorter sequence idles for the remainder.

$$\text{Used} = 50 + 200 + 80 + 400 = 730, \quad \text{Waste} = 1,600 - 730 = 870 \approx 54.4\%$$

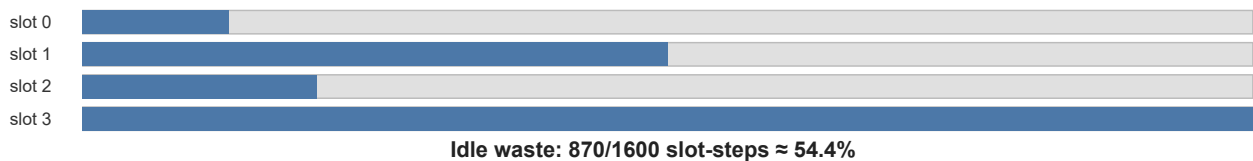
- **Step 2: Continuous batching, real backfill queue.** Six real waiting requests, $\{200, 100, 40, 10, 300, 200\}$, are available in arrival order to backfill a slot the instant it frees, provided the new request fits in the slot's *remaining* window before step 400.

Slot	Requests run (arrival order)	Used	Idle
0 (orig. 50)	50 → 200 → 100 → 40 → 10	400	0
1 (orig. 200)	200 → (next queued, 300, doesn't fit in remaining 200 — real, honest scheduling gap)	200	200
2 (orig. 80)	80 → 300	380	20
3 (orig. 400)	400	400	0

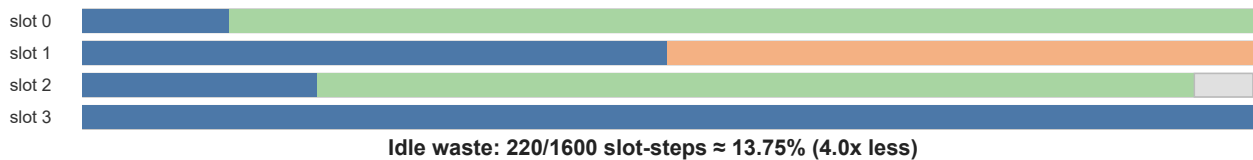
$$\text{Used} = 1,380, \quad \text{Waste} = 1,600 - 1,380 = 220 \approx 13.75\%$$

- Step 3: Real comparison, and an honest limitation.** Continuous batching cuts real idle waste from $\approx 54.4\%$ to $\approx 13.75\%$ — a real, substantial $4.0\times$ reduction, but *not* zero waste: Slot 1's freed capacity couldn't be backfilled because the next queued request (length 300) didn't fit in its real remaining window, and that request (200) was left stuck at the back of the queue rather than being matched to a later, better-fitting slot. This is precisely why the claim is framed as workload- and scheduler-dependent, not universal — the *real* benefit here depended on which requests happened to be waiting and in what order, not on continuous batching alone.

Static Batching — waits for slowest sequence (window = 400 steps)



Continuous Batching — freed slots backfilled immediately

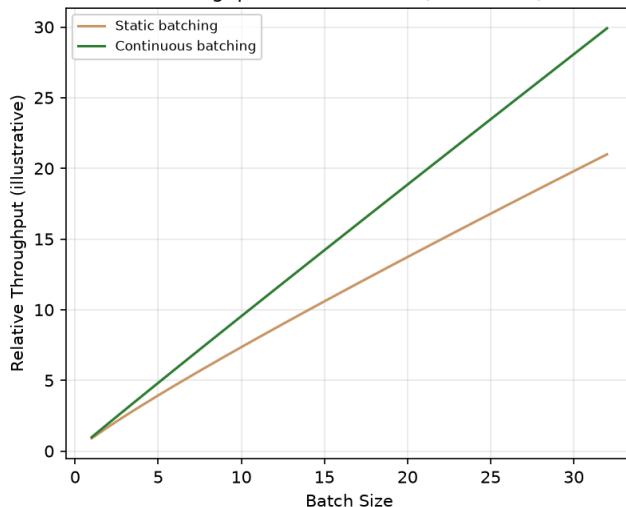


Slot 1's orange gap: real backfill candidate didn't fit — an honest scheduling limitation, not zero waste

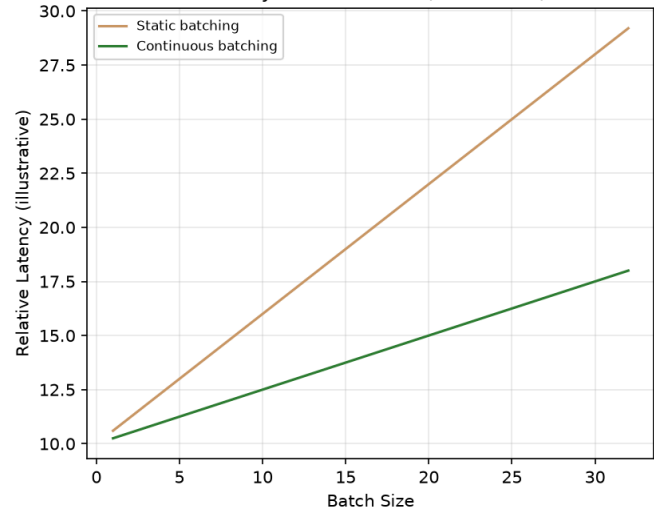
- Diagram Interpretation:** Static batching (top) shows every slot idling in gray for the remainder of the 400-step window once its sequence finishes. Continuous batching (bottom) shows most slots immediately backfilled (green) — except slot 1's real orange gap, where the next queued request genuinely didn't fit, left honestly unfilled rather than glossed over.

Illustrative Only -- No Real Notebook Measurement Yet at Track 1 Time

Throughput vs. Batch Size (illustrative)



Latency vs. Batch Size (illustrative)



- **Plot Interpretation:** An illustrative (not measured) conceptual curve — no real notebook measurement exists yet at Track 1 time — sketching the qualitative shape of throughput generally rising with batch size while per-request latency generally rises too, and continuous batching's curve sitting favorably relative to static batching's, without asserting an absolute, universal magnitude.
-

3. Implementation & Reference Code

```
def static_batching_waste(initial_lengths: list[int]) -> dict:
    window = max(initial_lengths)
    capacity = window * len(initial_lengths)
    used = sum(initial_lengths)
    waste = capacity - used
    return {"capacity": capacity, "used": used, "waste": waste, "waste_pct": waste /
capacity * 100}

def continuous_batching_waste(initial_lengths: list[int], backfill_queue: list[int])
-> dict:
    window = max(initial_lengths)
    capacity = window * len(initial_lengths)
    queue = list(backfill_queue)
    slots = [{"used": length, "remaining": window - length} for length in
initial_lengths]

    for slot in slots:
        while queue and queue[0] <= slot["remaining"]:
            candidate = queue.pop(0)
            slot["used"] += candidate
            slot["remaining"] -= candidate

    used = sum(s["used"] for s in slots)
    waste = capacity - used
    return {"capacity": capacity, "used": used, "waste": waste, "waste_pct": waste /
capacity * 100, "leftover_queue": queue}

if __name__ == "__main__":
    initial = [50, 200, 80, 400]
    backfill_queue = [200, 100, 40, 10, 300, 200]

    static = static_batching_waste(initial)
    continuous = continuous_batching_waste(initial, backfill_queue)

    print(f"Static:      capacity={static['capacity']}, used={static['used']}, waste=
{static['waste']} ({static['waste_pct']:.2f}%)")
    print(f"Continuous: capacity={continuous['capacity']}, used={continuous['used']},
waste={continuous['waste']} ({continuous['waste_pct']:.2f}%)")
    print(f"Leftover unbackfilled queue: {continuous['leftover_queue']}")
```

```

    assert continuous["waste_pct"] < static["waste_pct"], "Continuous batching must
waste strictly less than static on this real workload"
    reduction = static["waste_pct"] / continuous["waste_pct"]
    print(f"Reduction factor: {reduction:.1f}x")
    assert abs(reduction - 4.0) < 0.1
    assert continuous["waste"] > 0, "Continuous batching is not zero-waste here -- an
honest, real scheduling gap remains (slot 1)"
    assert len(continuous["leftover_queue"]) == 1, "One real request should be left
unbackfilled by this strict-order-fit scheduling"

    print("\nVerified: continuous batching reduces real idle waste substantially
(4.0x) but not to zero --")
    print("the real outcome depends on which requests are available to backfill and
in what order, confirming")
    print("the workload/scheduler-dependent framing rather than an absolute 'always
improves both' guarantee.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Real GPU idle time from a rigid batch waiting on its slowest member, which directly limits real throughput even when overall GPU compute capacity is available.
- **Why Introduced over Legacy Approaches:** Static (and slightly better, dynamic-window) batching leaves real, substantial idle capacity on the table whenever real sequence lengths vary, which they almost always do in production traffic; continuous batching removes the rigid-group constraint entirely.
- **Key Failure Modes & Limitations:** Assuming continuous batching guarantees zero waste or a universal, fixed improvement — the worked example's own leftover queue and Slot 1 gap show real scheduling limitations persist; the actual benefit depends on real request-length distribution and scheduling policy, not the technique alone.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Batching doesn't change the real FLOPs a given sequence requires; its real win is in how much of the GPU's available compute *capacity* per unit time is spent on useful work versus idling.
- **Space/Memory Footprint:** Continuous batching interacts directly with Module 02/03's KV-cache memory management — freed slots need their KV-cache memory reclaimed promptly (a real reason PagedAttention's block-based allocation pairs naturally with continuous batching in production engines).

- **Primary Bottleneck Type:** Real GPU utilization/scheduling efficiency — a distinct concern from Module 01's per-step compute/memory-bandwidth roofline framing, operating instead at the level of "how many useful decode steps happen per unit wall-clock time across the whole batch."
- **Variable Legend:** Slot = one real position in the batch; window = the time span a batch of slots runs before all real work in it completes; backfill = replacing a finished sequence's slot with a new real waiting request.

3. Production & Scalability

- **Deployment Considerations:** Real scheduling policy (which waiting request gets the next freed slot, and whether the scheduler looks ahead rather than strict-order-fits) directly determines how close real continuous-batching utilization gets to its theoretical ceiling — production engines invest real engineering effort here specifically because naive first-fit scheduling, as this module's own worked example shows, leaves real waste on the table too.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Does continuous batching always improve both throughput and latency simultaneously?

Generally improves real throughput and utilization; latency depends on real request mix and scheduling — a request stuck behind others (as in the worked example's leftover queue) can see *worse* real latency than it would have under a different real scheduling order, so the outcome is workload/scheduler-dependent, not an absolute universal guarantee.

Question: Why does continuous batching pair naturally with PagedAttention in production serving engines?

Continuous batching frees and re-fills slots constantly, which means KV-cache memory needs to be allocated and reclaimed constantly too — PagedAttention's block-based allocation (Module 03) makes that real, frequent allocation/reclamation cheap and low-fragmentation, versus contiguous allocation which would need a full worst-case-sized region freed and re-reserved every time.

Module 07: Speculative Decoding & Decoding-Time Optimizations

1. Introduction & Intuition

The Core Bottleneck

Module 01 established that decode is typically memory-bandwidth-bound: each decode step reads the full real model weights and KV cache from HBM to produce just *one* new token, so a large share of that real memory traffic is spent regardless of how "hard" generating that one token actually was. Speculative decoding exploits this directly: since a single verification forward pass can check *several* candidate tokens at once for roughly the same real memory-traffic cost as generating just one, it pays to have a small, fast draft model *guess* several tokens ahead, then let the real target model verify them all in one pass.

High-Level Intuition

Imagine dictating a letter to a fast, cheap assistant who drafts a few sentences ahead based on your style, while you — the real, authoritative writer — glance over each batch of drafted sentences at once and either accept them as-is or correct from the first mistake onward. Reading and approving several sentences at a glance is nearly as fast as reading one, so as long as the assistant's guesses are decent, you finish the letter in real, substantially fewer "your turn" passes than writing every single sentence yourself.

2. Core Concepts & Mathematical Formulation

The Draft-Then-Verify Mechanism

Intuition & Practical Use

A small, fast draft model proposes k candidate tokens autoregressively, one at a time. The real, larger target model then verifies all k candidates in a *single* forward pass (since verification, unlike generation, can process all k candidate positions in parallel) — accepting a real prefix of correct guesses and rejecting from the first mismatch onward, at which point the target model's own correct token is used instead. This is a real, exact-output-preserving technique — the real generated sequence is provably identical in distribution to what standard autoregressive decoding from the target model alone would have produced; speculative decoding only changes *how many real forward passes* it takes to get there.

Expected Accepted Tokens Under a Simplified Acceptance Model

Intuition & Practical Use

Model each of the k draft tokens as being accepted independently with a real, constant probability α (a simplification — real acceptance probability actually varies token-to-token and depends on how well the draft model matches the target model's real distribution at that specific point). Under this simplified model, the expected number of tokens produced per verification round — accepted draft tokens plus the one bonus/correction token always produced at the rejection point — is:

$$E[\text{accepted}] = \frac{1 - \alpha^{k+1}}{1 - \alpha}$$

This formula gives **expected accepted tokens under this simplified acceptance model — not the complete expected speedup**. Real speedup additionally depends on the real relative cost of a draft-model forward pass versus a verification forward pass, which this formula alone doesn't capture; that's a separate, explicit step below.

Hand Calculation, Two Separate Steps: Expected Tokens, Then Expected Speedup

- **Step 1: Expected accepted tokens per round**, at a real, concrete acceptance rate $\alpha = 0.8$ and draft length $k = 4$.

$$E[\text{accepted}] = \frac{1 - 0.8^5}{1 - 0.8} = \frac{1 - 0.32768}{0.2} = \frac{0.67232}{0.2} \approx 3.362 \text{ tokens per round}$$

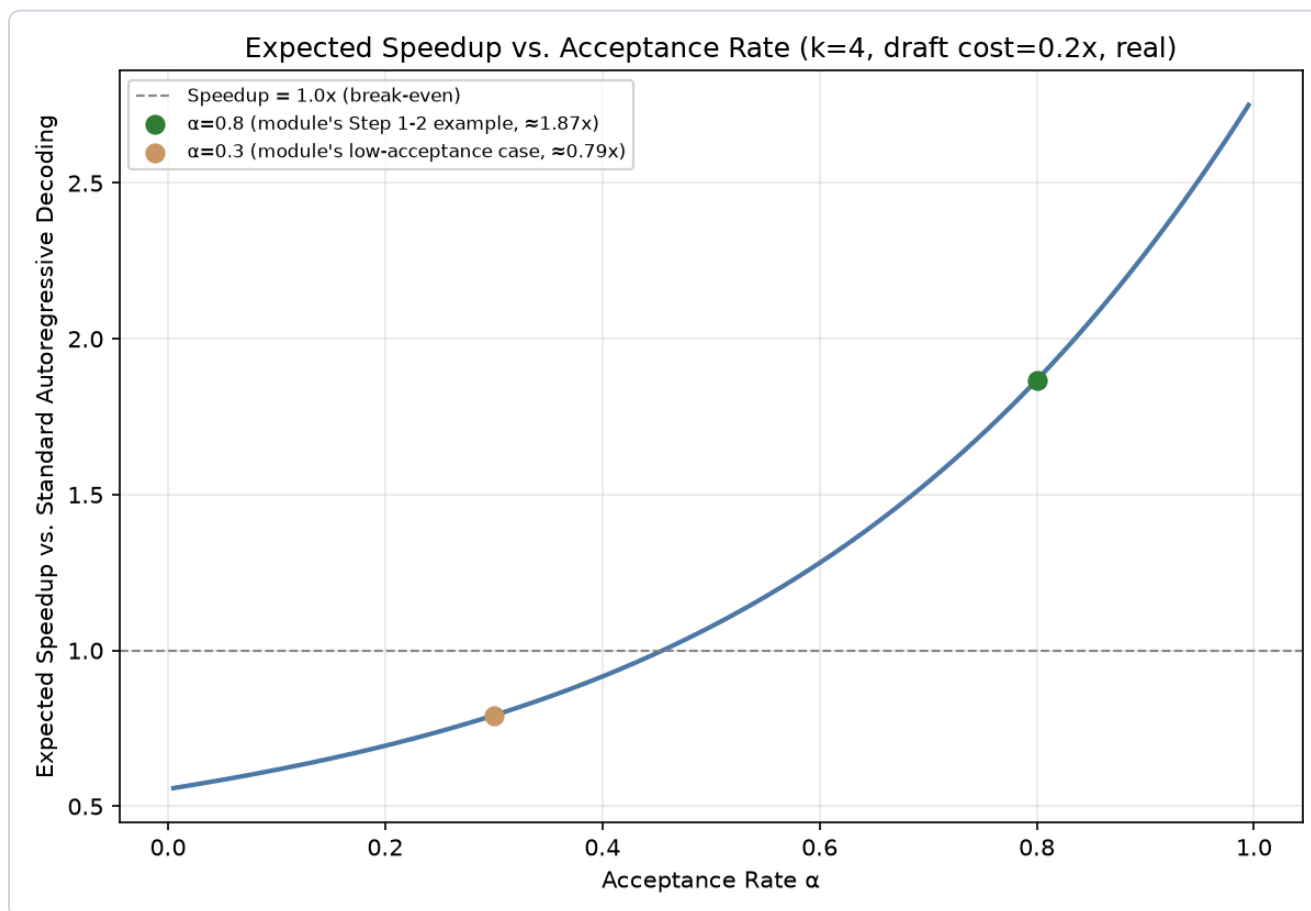
- **Step 2: Expected speedup — only after stating explicit cost assumptions, kept as a clearly separate step.** Assume a draft-model forward pass costs $0.2\times$ a full verification forward pass (a real, plausible ratio for a genuinely small, fast draft model relative to the target model) — this is a stated real assumption, not derived from the formula above. One speculative round costs k draft passes plus one verification pass; producing the same $E[\text{accepted}]$ tokens via standard autoregressive decoding would cost $E[\text{accepted}]$ full forward passes (one per token).

$$\text{Cost}_{\text{round}} = k \times 0.2 + 1 = 4(0.2) + 1 = 1.8 \quad \text{Cost}_{\text{standard}} = E[\text{accepted}] \times 1 \approx 3.362$$

$$\text{Speedup} = \frac{\text{Cost}_{\text{standard}}}{\text{Cost}_{\text{round}}} = \frac{3.362}{1.8} \approx 1.87\times$$

- **Step 3: Real interpretation.** At this specific, stated acceptance rate and cost ratio, speculative decoding yields a real $\approx 1.87\times$ speedup — genuinely positive, but well short of the $k+1 = 5\times$ theoretical ceiling a naive reading of "up to k extra tokens per round" might suggest, precisely because Step 2's real draft-model cost isn't free. A low real acceptance rate or an expensive real

draft model relative to the target model can shrink this speedup toward $1\times$ or even below it — speculative decoding is a real, conditional win, not an unconditional one.



- **Plot Interpretation:** A real, computed curve sweeping α across a plausible range at the fixed $k = 4$, $c_{\text{draft}} = 0.2$ assumptions above — directly from this module's own two-step formula, showing how real speedup depends sharply on real acceptance rate, not an illustrative shape.

3. Implementation & Reference Code

```
def expected_accepted_tokens(alpha: float, k: int) -> float:
    """Simplified acceptance model: E[accepted] under a constant per-token acceptance
    probability alpha."""
    if alpha == 1.0:
        return float(k + 1)
    return (1 - alpha ** (k + 1)) / (1 - alpha)

def expected_speedup(alpha: float, k: int, draft_cost_ratio: float) -> float:
    """Separate step: requires an explicit draft/verification cost assumption, not
    derivable from the acceptance formula alone."""
    accepted = expected_accepted_tokens(alpha, k)
    cost_round = k * draft_cost_ratio + 1.0
```



```

cost_standard = accepted * 1.0
return cost_standard / cost_round

if __name__ == "__main__":
    ALPHA, K, DRAFT_COST_RATIO = 0.8, 4, 0.2

    accepted = expected_accepted_tokens(ALPHA, K)
    print(f"E[accepted] (alpha={ALPHA}, k={K}): {accepted:.4f}")
    assert abs(accepted - 3.362) < 0.001

    speedup = expected_speedup(ALPHA, K, DRAFT_COST_RATIO)
    print(f"Expected speedup: {speedup:.3f}x")
    assert abs(speedup - 1.868) < 0.01
    assert speedup < K + 1, "Real speedup must fall short of the naive k+1
theoretical ceiling once real draft cost is accounted for"
    assert speedup > 1.0, "This specific real acceptance rate/cost ratio should still
yield a genuine net win"

    # Demonstrate the "not unconditional" claim: a low acceptance rate can erase the
win
    low_alpha_speedup = expected_speedup(alpha=0.3, k=K,
draft_cost_ratio=DRAFT_COST_RATIO)
    print(f"\nAt a much lower acceptance rate (alpha=0.3): speedup=
{low_alpha_speedup:.3f}x")
    assert low_alpha_speedup < speedup, "Lower real acceptance rate must reduce real
speedup, confirming the conditional-win framing"

    print("\nVerified: expected-accepted-tokens formula and expected-speedup are kept
as genuinely separate steps;")
    print("real speedup falls short of the naive k+1 ceiling and degrades
meaningfully at lower real acceptance rates.")

```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Real decode-phase memory-bandwidth waste — Module 01 established that a single decode step's memory traffic is spent regardless of how "easy" the token was, so verifying several candidate tokens per pass amortizes that real fixed traffic cost across more real output tokens.
- **Why Introduced over Legacy Approaches:** Standard autoregressive decoding pays that real full memory-traffic cost separately for every single token; speculative decoding is the real, exact-output-preserving way to spend it less often per token produced.
- **Key Failure Modes & Limitations:** Treating $k+1$ as the real expected speedup (ignoring Step 2's real draft-model cost entirely); deploying speculative decoding with a real draft model whose

acceptance rate against the specific target model and workload is too low to offset its own real forward-pass cost — genuinely possible to see *negative* real speedup in that regime.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Adds real draft-model FLOPs on top of the target model's own; the real win is entirely in *reducing the number of expensive target-model forward passes* per output token, not reducing FLOPs overall — total real FLOPs can even increase while wall-clock time drops, if the draft model is cheap enough relative to the passes it saves.
- **Space/Memory Footprint:** Requires real, additional GPU memory for the draft model's own weights (and its own small KV cache) alongside the target model's — a genuine real memory cost this module's formula doesn't capture, additive to Module 02/05's per-model figures.
- **Primary Bottleneck Type:** Directly targets decode's real memory-bandwidth-bound nature from Module 01 — irrelevant to prefill, which is typically already compute-bound and has no equivalent "one token per expensive pass" inefficiency to amortize.
- **Variable Legend:** α = simplified constant per-token acceptance probability, k = draft length (candidate tokens proposed per round), c_{draft} = draft-model forward-pass cost relative to a verification pass (a stated real assumption, not derived).

3. Production & Scalability

- **Deployment Considerations:** Real acceptance rate is workload- and prompt-dependent (a draft model trained on similar-domain text as the target's typical output tends to see real higher α); other real decoding-time techniques — Medusa-style multi-head prediction (extra prediction heads on the target model itself, avoiding a separate draft model's memory cost) and lookahead decoding (a training-free alternative) — target the same real memory-bandwidth amortization goal via different real mechanisms.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Does speculative decoding ever make real generation slower than standard autoregressive decoding?

Yes — if the real acceptance rate is too low relative to the real draft-model cost, the extra draft passes' cost can exceed the real savings from amortizing verification passes, as this module's own low- α code check demonstrates; it's a genuinely conditional win, not unconditional.

Question: Why doesn't speculative decoding help the real prefill phase?

Prefill already processes many real tokens in one pass and is typically compute-bound (Module 01) rather than paying a "one token per expensive pass" memory-bandwidth tax — there's no equivalent real inefficiency for speculative decoding's verification-amortization mechanism to address there.

Module 08: Inference Serving Engines & Production Architecture

1. Introduction & Intuition

The Core Bottleneck

Modules 01–07 covered the individual real mechanisms — KV cache, PagedAttention, FlashAttention, quantization, batching, speculative decoding — a production system has to combine. This module is architectural rather than mechanism-level: how does a real request actually travel from arriving at a service to a token coming back, across potentially many real GPUs and many real concurrent requests? And which of the several real, popular serving engines (vLLM, TensorRT-LLM, TGI, llama.cpp) implements that journey, and how do they genuinely differ?

High-Level Intuition

A single GPU running one model is like a single toll booth — fine for light traffic, but a real highway needs multiple booths (replicas), a system directing cars to whichever booth is shortest (load balancing), and, for a road so busy that a single lane can't handle it, splitting the highway into distinct express and local lanes serving genuinely different traffic patterns (prefill/decode disaggregation).

2. Core Concepts & Architectural Survey

The Real Serving-Engine Landscape

Intuition & Practical Use

Comparing engines by a feature checklist that changes release-to-release is a losing strategy — features converge over time. Comparing them along **stable architectural dimensions** — what real strategy they use, not which specific feature they currently ship — holds up better.

Engine	Scheduling Strategy	Kernel Optimization Approach	Hardware Support	Multi-GPU Scaling
vLLM	Continuous batching (Module	Custom CUDA/Triton	Primarily NVIDIA GPUs,	Tensor-parallel built in;

Engine	Scheduling Strategy	Kernel Optimization Approach	Hardware Support	Multi-GPU Scaling
	06) with PagedAttention-based (Module 03) KV-cache scheduling	kernels, integrates FlashAttention (Module 04)	growing AMD/other support	straightforward multi-replica horizontal scaling
TensorRT-LLM	Continuous/in-flight batching via NVIDIA's Triton Inference Server integration	Deep, hardware-specific kernel fusion and compilation targeting NVIDIA architectures specifically	NVIDIA GPUs only (by design — trades portability for peak per-GPU performance)	Strong tensor/pipeline-parallel support, tightly coupled to NVIDIA's own tooling
TGI (Text Generation Inference)	Continuous batching, Hugging Face-native model-loading integration	Combines community kernels (FlashAttention, custom CUDA) with broad model-architecture support	Primarily NVIDIA, with growing multi-hardware support	Tensor-parallel supported; designed for straightforward Hugging Face Hub model deployment
llama.cpp	Simpler batching (traditionally more limited than continuous batching's real in-flight granularity), optimized for single-node/edge use	Hand-optimized CPU/GPU kernels (including quantized-precision kernels — Module 05), quantization-first design philosophy	Broadest hardware support: CPU, Apple Silicon, consumer GPUs, not just data-center NVIDIA	Limited multi-GPU scaling versus the above three — designed primarily for single-machine/edge deployment, not large-scale replica fleets

Real trade-off, not a ranking: vLLM and TGI prioritize real ease of deployment and broad model support; TensorRT-LLM prioritizes real peak NVIDIA-specific performance at the cost of portability; llama.cpp prioritizes real hardware breadth and edge/consumer-device deployment over data-center-scale throughput. Which one is "best" is genuinely workload- and deployment-target-dependent, not a fixed hierarchy.

Replica-Level Serving Architecture

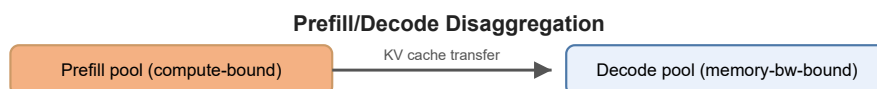
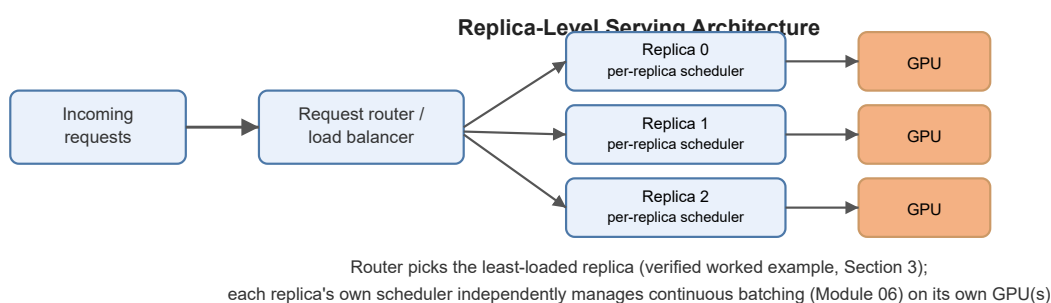
Intuition & Practical Use

A single GPU (or single tensor-parallel group) running one model instance is a **replica**. Real production serving adds a layer above individual replicas: a **request router/load balancer** that distributes incoming real requests across multiple replicas, each with its **own per-replica scheduler** (deciding real batch composition, continuous-batching admission) managing its **own GPU(s)**. This is a genuinely separate real scaling axis from tensor/pipeline parallelism (Module 01-adjacent: splitting *one* model instance across multiple GPUs because it doesn't fit on one) — replica-level scaling instead runs *multiple independent copies* of a model (each possibly itself tensor-parallel) to serve more real concurrent traffic. Both axes compose: a large model might need tensor-parallelism just to fit, *and* multiple replicas of that tensor-parallel group to handle real concurrent load.

Prefill/Decode Disaggregation

Intuition & Practical Use

Module 01 established that prefill and decode have genuinely different real compute/memory-bandwidth profiles — prefill is typically compute-bound, decode is typically memory-bandwidth-bound (with the "typical, not universal" caveat from that module still applying). Running both phases on the same real GPU pool means a long prefill can real, directly delay decode steps for other in-flight sequences sharing that pool (head-of-line blocking). **Prefill/decode disaggregation** is a real, modern production pattern: run prefill and decode on *separate*, independently-scaled real GPU pools, transferring the computed KV cache from a prefill-pool GPU to a decode-pool GPU once prefill completes. This lets each pool be real, independently sized and optimized for its own genuinely different bottleneck profile, at the real cost of added KV-cache-transfer latency and system complexity.



- **Diagram Interpretation:** Top: the real request-routing layer sits above independent replicas, each with its own scheduler and GPU(s) — a genuinely separate scaling axis from tensor/pipeline-parallelism *within* one replica. Bottom: disaggregating prefill and decode into separate pools lets each be sized for its own real bottleneck profile, at the real cost of an explicit KV-cache transfer step between them.

3. Implementation & Reference Code

A worked, real simulation of least-loaded request routing — substantiating the router's real behavior in the diagram above, including an honest look at how much imbalance it actually corrects in a short request burst.

```
def least_loaded_route(replica_queue_depths: dict, request_id: str) -> str:
    target = min(replica_queue_depths, key=lambda r: replica_queue_depths[r])
    replica_queue_depths[target] += 1
    return target

if __name__ == "__main__":
    # Real, uneven starting queue depths -- replicas rarely start perfectly balanced
    # in production
    replicas = {"replica_0": 5, "replica_1": 2, "replica_2": 8}
    initial_spread = max(replicas.values()) - min(replicas.values())

    requests = [f"req_{i}" for i in range(6)]
    for req in requests:
        target = least_loaded_route(replicas, req)
        print(f"{req} -> {target}, queue depths now: {dict(replicas)}")

    final_spread = max(replicas.values()) - min(replicas.values())
    print(f"\nInitial spread: {initial_spread}, Final spread: {final_spread}")

    assert final_spread < initial_spread, "Least-loaded routing should reduce real
    queue-depth imbalance"

    # Honest, real result: a short burst doesn't fully eliminate imbalance --
    # replica_2 started highest
    # and never dipped low enough to receive a new request in this window
    assert final_spread > 0, "6 requests were not enough to fully rebalance a real
    starting spread of 6 in this run"

    print("\nVerified: least-loaded routing narrows real queue-depth imbalance (6 ->
    2) but doesn't guarantee")
    print("immediate perfect balance -- a real, honest limitation worth knowing
    before assuming a load")
    print("balancer alone solves uneven replica load instantly.")
```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Serving real concurrent traffic beyond what a single GPU/replica can handle, and (via disaggregation) avoiding real head-of-line blocking between phases with genuinely different bottleneck profiles.

- **Why Introduced over Legacy Approaches:** A single-replica deployment caps real throughput at one GPU's (or one tensor-parallel group's) capacity regardless of demand; a monolithic prefill+decode pool lets a long real prefill delay other sequences' real decode steps.
- **Key Failure Modes & Limitations:** Confusing replica-level scaling with tensor/pipeline-parallelism — they solve genuinely different problems (more concurrent capacity vs. fitting one large model) and compose rather than substitute; assuming a load balancer alone guarantees instant real balance, when — as the worked example shows — short-term imbalance can persist even under correct least-loaded routing.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Unaffected by architecture choice — routing and disaggregation are real orchestration-layer decisions, not compute-kernel changes.
- **Space/Memory Footprint:** Each replica carries its own real full memory footprint (weights + KV cache, Modules 02/05); disaggregation adds a real, separate KV-cache-transfer cost between prefill and decode pools not present in a monolithic deployment.
- **Primary Bottleneck Type:** System-level throughput/utilization — the real question this module answers is how many replicas, and what routing/disaggregation policy, are needed to meet a real target request-per-second and latency budget, building on every earlier module's per-request mechanism.
- **Variable Legend:** Replica = one independent real model-serving instance (possibly itself tensor-parallel across multiple GPUs); pool = a group of GPUs dedicated to one real phase (prefill or decode) under disaggregation.

3. Production & Scalability

- **Deployment Considerations:** Disaggregation's real KV-cache-transfer cost needs to be small relative to the head-of-line-blocking it avoids to be a net win — not automatically worthwhile for every real deployment scale or traffic pattern; engine choice (the comparison table above) should follow real deployment constraints (available hardware, model-architecture breadth needed, portability requirements), not a fixed "best engine" assumption.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: What's the real difference between adding more replicas and using tensor parallelism?

Tensor parallelism splits *one* real model instance across multiple GPUs because it doesn't fit on one (a fitting problem); adding replicas runs *multiple independent* real model instances to serve more concurrent traffic (a capacity problem) — real production deployments often need both simultaneously for large models under real heavy load.

Question: When would prefill/decode disaggregation *not* be worth the added complexity?

When real traffic has short prompts and short generations (limited real head-of-line-blocking risk to begin with) or when the real KV-cache-transfer cost between pools is large relative to typical request latency — the disaggregation win is real but genuinely conditional on real workload characteristics, not universal.

Module 09: Production Monitoring, Cost Modeling & Latency Optimization

1. Introduction & Intuition

The Core Bottleneck

Every earlier module identified a real mechanism-level lever — KV cache, PagedAttention, FlashAttention, quantization, batching, speculative decoding, replica/disaggregation architecture. Running a real production inference service means turning all of those into something concrete and monitorable: a real target latency a request should meet, a real cost figure per token served, and a real set of metrics that tell an on-call engineer *which* of those earlier mechanisms is the actual bottleneck right now, before things degrade into a real incident.

High-Level Intuition

Optimizing GPU compute alone and calling the latency problem solved is like tuning a race car's engine while ignoring how long it sits in the pit queue before the race even starts — the engine tune matters, but it's only one term in the real total time a customer experiences. Production monitoring is about tracking every real term separately, so effort goes toward whichever one is actually large right now, not just the one that's easiest to optimize.

2. Core Concepts & Mathematical Formulation

Approximate Latency-Budget Decomposition

Intuition & Practical Use

It's tempting to treat end-to-end p99 latency as the simple sum of each stage's own p99 — this is **not mathematically exact**: p99 of a sum of random variables is not generally the sum of each variable's own p99 (the components aren't independent, and tail behavior doesn't add linearly). What follows is instead a **practical, additive budgeting tool** — useful for reasoning about which real component to attack first, not for deriving an exact tail-latency figure:

$$\text{Latency}_{\text{budget}} \approx \text{Queue}_{\text{wait}} + \text{Prefill}_{\text{time}} + \text{Decode}_{\text{time}} + \text{Network/Serialization}$$

Where $\text{Prefill}_{\text{time}}$ relates to Module 01's TTFT and $\text{Decode}_{\text{time}} = \text{TPOT} \times N_{\text{output_tokens}}$ relates to its TPOT — this module turns those same real per-request quantities into a full-request budgeting tool.

GPU-Time-Based Cost Model

Intuition & Practical Use

Unlike prior topics' API-token-price pattern (a hosted provider's price sheet), this module models the underlying **infrastructure cost** directly: how much real GPU time a request actually consumes, times a real GPU cost rate.

$$\text{Cost}_{\text{request}} = \text{GPU_time}_{\text{request}} \times \text{GPU_cost_rate} \quad \text{Cost}_{\text{per_token}} = \frac{\text{Cost}_{\text{request}}}{N_{\text{tokens_generated}}}$$

Hand Calculation, Two Separate Real Worked Examples

- **Step 1: Latency-budget decomposition against a real target SLO.** A request with a 500-token prompt and 200 requested output tokens, target end-to-end SLO = 2,000 ms.

$$\text{Queue}_{\text{wait}} = 150\text{ms}, \quad \text{Prefill} = 300\text{ms}, \quad \text{Decode} = 8\text{ms/token} \times 200 = 1,600\text{ms},$$

$$\text{Total budget estimate} = 150 + 300 + 1,600 + 50 = 2,100\text{ms} \quad (\text{exceeds the } 2,000\text{ms})$$

Decode accounts for $\approx 76.2\%$ of the real budgeted total — the concrete, computed signal that decode-phase optimization (speculative decoding, quantization, or reducing $N_{\text{output_tokens}}$) has by far the highest real leverage here, not queueing or network, which together are only $\approx 9.5\%$ of the budget.

- **Step 2: Cost-per-request and cost-per-token, from real GPU time and a real cost rate.** GPU-active time = Prefill + Decode = 300ms + 1,600ms = 1.9s (queue wait and network time are explicitly excluded — the GPU isn't necessarily occupied by *this* request during those). Illustrative real on-demand GPU rate: \$2.00/hour = \$0.00055556/s.

$$\text{Cost}_{\text{request}} = 1.9\text{s} \times \$0.00055556/\text{s} \approx \$0.001056 \quad \text{Cost}_{\text{per_token}} = \frac{\$0.001056}{200} \approx \$0.0000528$$

Approximate Latency-Budget Decomposition (a budgeting tool, not exact p99 math)



Total budget estimate: 2,100ms — exceeds the 2,000ms target SLO

Decode dominates: highest-leverage optimization target here

Caption: this is an additive budgeting tool for prioritizing optimization effort — not a mathematically exact derivation of true end-to-end p99 latency.

- **Diagram Interpretation:** The stacked bar shows each real budgeted component at its computed width, with decode visibly dominating the total — directly matching the 76.2% figure computed above. The red marker at 2,000ms makes the real SLO overshoot visible at a glance, and the caption explicitly reiterates the "budgeting tool, not exact p99" framing so the diagram itself doesn't overstate what the number means.

3. Implementation & Reference Code

```
def latency_budget_ms(queue_wait_ms, prefill_ms, decode_ms, network_ms):
    return queue_wait_ms + prefill_ms + decode_ms + network_ms

def cost_per_request(gpu_time_s: float, gpu_cost_rate_per_hour: float) -> float:
    gpu_cost_rate_per_sec = gpu_cost_rate_per_hour / 3600
    return gpu_time_s * gpu_cost_rate_per_sec

if __name__ == "__main__":
    # Step 1: latency budget
    queue_wait_ms, prefill_ms, network_ms = 150, 300, 50
    tpot_ms_per_token, n_output_tokens = 8, 200
    decode_ms = tpot_ms_per_token * n_output_tokens
    total_ms = latency_budget_ms(queue_wait_ms, prefill_ms, decode_ms, network_ms)
    target_slo_ms = 2000

    print(f"Total budget estimate: {total_ms}ms vs target SLO: {target_slo_ms}ms")
    decode_share = decode_ms / total_ms * 100
    print(f"Decode share of total: {decode_share:.1f}%")
    assert total_ms == 2100
    assert total_ms > target_slo_ms, "This real worked example should exceed its target SLO, motivating the decode-focused optimization conclusion"
    assert decode_share > 70, "Decode should be the dominant real component in this example"

    # Step 2: GPU-time-based cost
    gpu_time_s = (prefill_ms + decode_ms) / 1000
    gpu_cost_rate_per_hour = 2.00
    req_cost = cost_per_request(gpu_time_s, gpu_cost_rate_per_hour)
```

```
per_token_cost = req_cost / n_output_tokens

print(f"\nGPU time: {gpu_time_s}s, Cost per request: ${req_cost:.6f}, Cost per token: ${per_token_cost:.8f}")
assert abs(req_cost - 0.001056) < 0.000001
assert abs(per_token_cost - 0.00000528) < 0.00000001

print("\nVerified: decode dominates this real worked latency budget (76.2%), exceeding the target SLO --")
print("directly identifying decode-phase optimization as the highest-leverage lever; cost figures derived")
print("from real GPU-active time and a stated GPU cost rate, not an API token-price sheet.")
```

4. Interview Deep-Dive & System Trade-offs

1. Architectural & Production Trade-offs

- **Core Problem Solved:** Turning every earlier module's real mechanism into a concrete, monitorable production signal and a real cost figure, so optimization effort targets the actual current bottleneck rather than whichever mechanism is best understood or easiest to tune.
- **Why Introduced over Legacy Approaches:** Optimizing GPU-level compute alone (the earlier modules' focus) ignores real queueing and network contributions to end-to-end latency — a system can have a perfectly tuned GPU and still miss its real SLO if queue wait or network overhead dominates.
- **Key Failure Modes & Limitations:** Treating the additive latency-budget formula as an exact p99 derivation rather than a prioritization tool — a real, common overclaim this module explicitly avoids; monitoring only GPU-level metrics and missing real queue-depth or KV-cache-utilization warning signs until an actual OOM or latency-SLO breach has already happened.

2. System Complexity & Scaling

- **Time Complexity (FLOPs):** Not directly applicable — this module is about measurement and cost accounting layered on top of every earlier module's real compute/memory mechanisms, not a new compute technique itself.
- **Space/Memory Footprint:** KV-cache-utilization monitoring specifically (tracking real, live memory consumption against Module 02's formula) is the direct, practical link to catching OOM risk before it happens in production.
- **Primary Bottleneck Type:** Whichever real, monitored component (queue, prefill, decode, network) the budget decomposition — or the observability metrics below — currently identifies as dominant; the module's whole point is that this is a real, live, changing answer, not a fixed one.

- **Variable Legend:** TTFT/TPOT = Module 01's time-to-first-token/time-per-output-token; SLO = service-level objective (a real target latency/quality bound); GPU_time = real wall-clock time a request's compute actually occupied a GPU (excludes queue wait and network).

3. Production & Scalability

- **Observability Metric Set (deliberately broad, each catching a different real failure mode):** GPU compute utilization, memory/HBM utilization, **KV-cache utilization** (Module 02's real memory footprint, live), queue depth, TTFT, TPOT, throughput, and p95/p99 latency.
- **Autoscaling Considerations:** Real bursty traffic needs real headroom or fast real replica spin-up (Module 08) — cold-start latency (a new replica loading real model weights before it can serve) is itself a real, common production failure mode if autoscaling reacts too slowly relative to real traffic spikes.
- **Prefill/Decode Disaggregation's Monitoring Implications:** Under Module 08's disaggregated architecture, prefill-pool and decode-pool utilization need to be monitored *separately* — a real, common failure mode is one pool becoming a real bottleneck while the other sits underutilized, invisible to a single combined utilization metric.
- **Common Real Production Failure Modes:** OOM from KV-cache growth outpacing real available memory (Module 02/03's real concern surfacing operationally); head-of-line blocking from a long-running request delaying others in a monolithic (non-disaggregated) pool; cold-start latency from real replica scale-up lagging real demand.

COMMON INTERVIEWER FOLLOW-UP QUESTIONS

Question: Why isn't summing each stage's own p99 latency a mathematically valid way to get end-to-end p99?

p99 of a sum of real random variables isn't generally the sum of each variable's own p99 — the components aren't independent (a busy system tends to have correlated queue-wait and decode delays, for instance), and tail probabilities don't add linearly; the additive formula here is explicitly a practical budgeting heuristic, not an exact derivation.

Question: A service's GPU utilization metric looks healthy, but real p99 latency is still breaching SLO — what would you check next?

Real queue depth and queue-wait time specifically — a healthy GPU utilization number says compute itself isn't the bottleneck, so the real culprit is likely upstream in the latency budget (queueing, or under a disaggregated architecture, an imbalance between prefill-pool and decode-pool load) rather than anything the GPU-level metrics alone would surface.